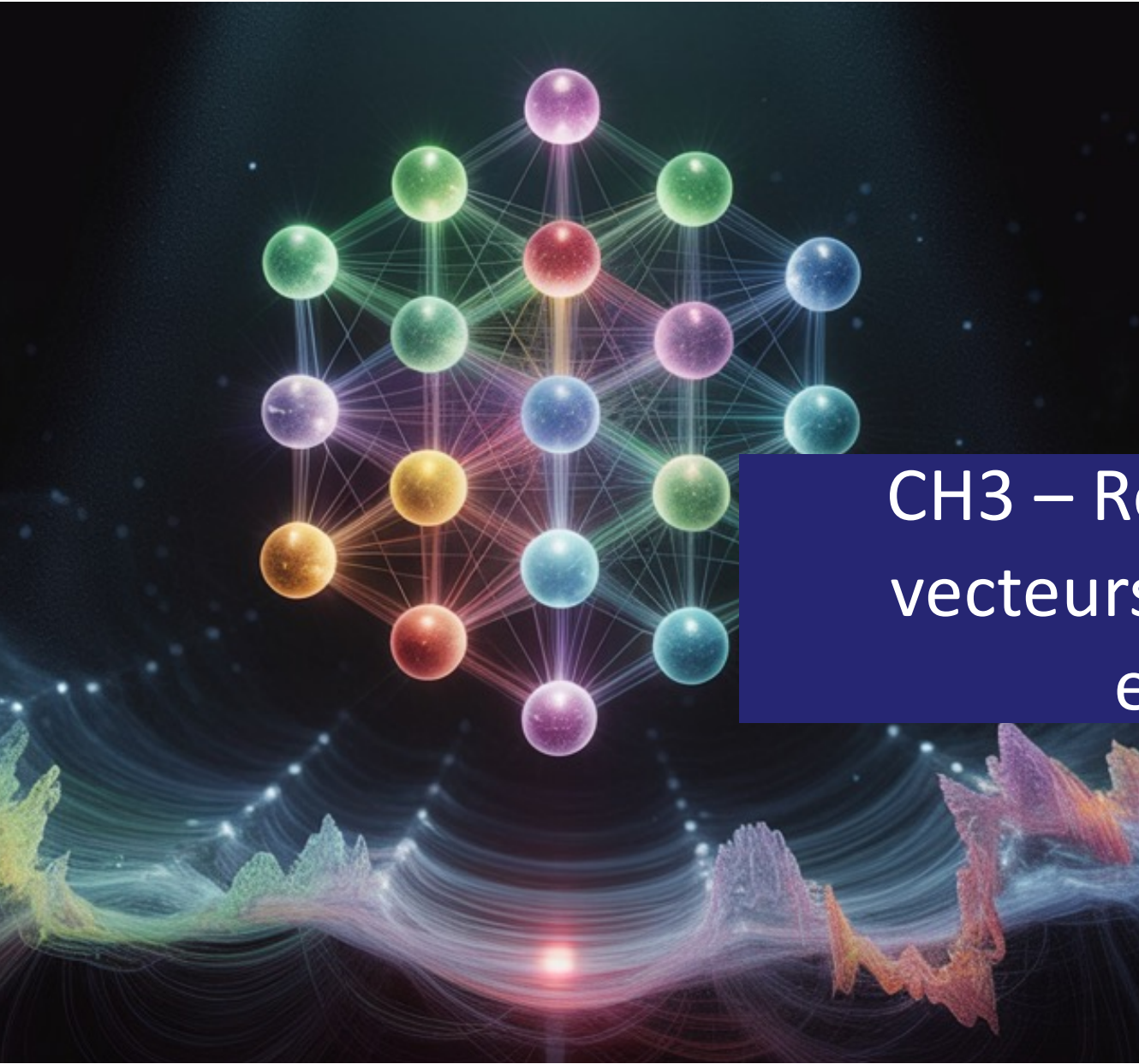




UNIVERSITE DE CORSE
Master Informatique
parcours DFS et DE
2^{ème} année
2025-2026

CH3 – Représenter par des vecteurs : fondements des embeddings

Evelyne VITTORI
vittori_e@univ-corse.fr

Partie I - Représenter le monde : des symboles aux vecteurs



CH1 – Représenter
le monde en IA : un
enjeu central



CH2 –
Représentations en
IA symbolique : des
origines aux
ontologies



CH3 – Représenter
par des vecteurs :
fondements des
embeddings



CH4 – Des
embeddings aux BD
Vectorielles



Objectifs du chapitre

- Poser les **bases mathématiques** nécessaires
- Identifier les limites des représentations simples (one-hot, TF-IDF)
- Introduire la notion d'**embedding** (vecteurs denses appris)
- Découvrir les modèles **statiques** et leurs limites
- Explorer les modèles **contextuels**

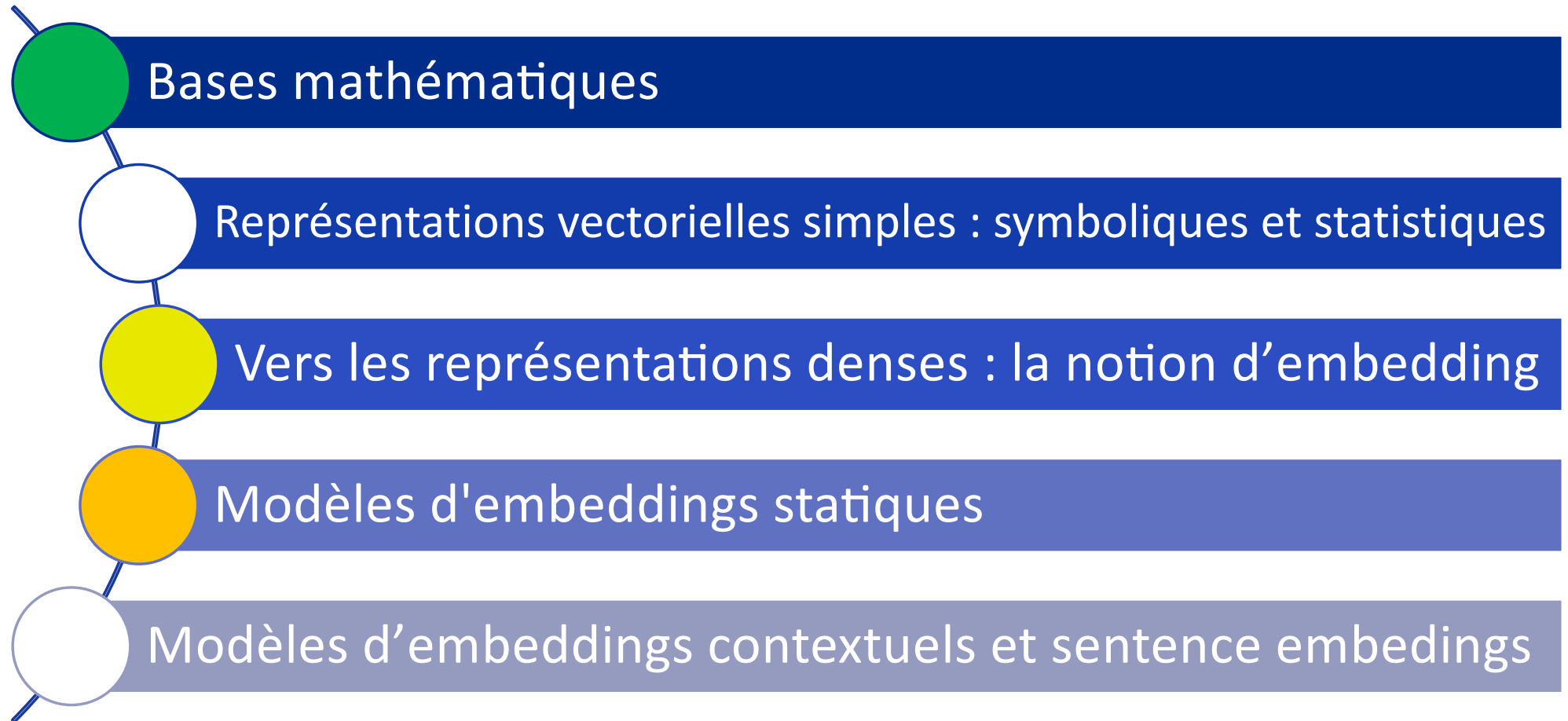


Activités guidées :

- Manipuler des Représentations simples
- Générer et comparer des embeddings avec différents modèles

Word2Vec, FastText, BERT, SentenceBERT

CH3 – Représenter par des vecteurs : Fondements des embeddings



La vectorisation n'est pas née avec l'IA



Premiers vecteurs : purements syntaxiques sans aucune ambition sémantique

- Besoins pratiques dans deux domaines :



Recherche d'information (IR)



Statistique textuelle

- Un usage fondamental



Rendre les mots et les documents manipulables numériquement

Pourquoi représenter en vecteurs en IA ?

Uniformiser les données



- Tout devient un vecteur dans un espace $\mathbb{R}^d$:
- un mot , une image , un son $\rightarrow$ vecteur

Faciliter le calcul



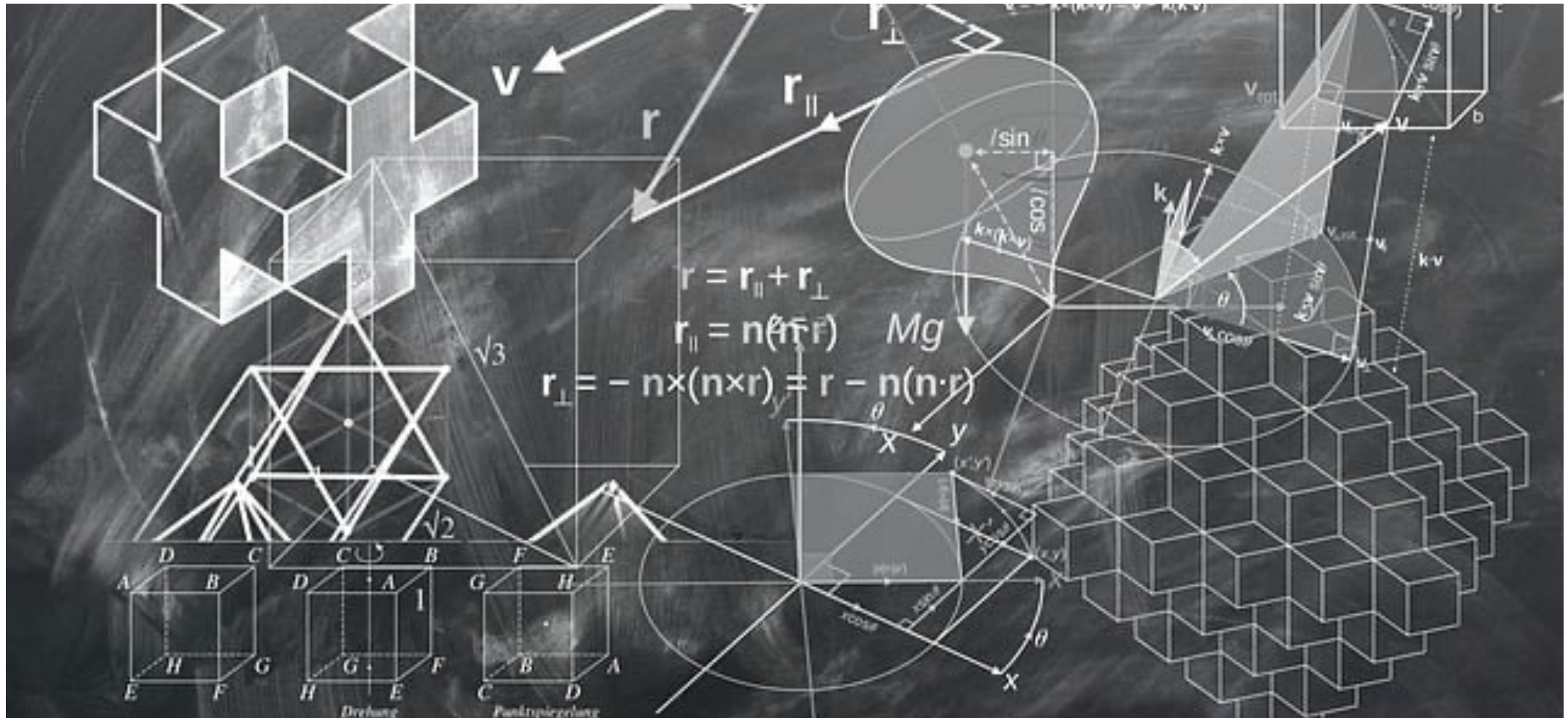
- mesurer la similarité (ex. cosinus, distance euclidienne)
- faire des opérations algébriques

Rendre possible
l'apprentissage
automatique



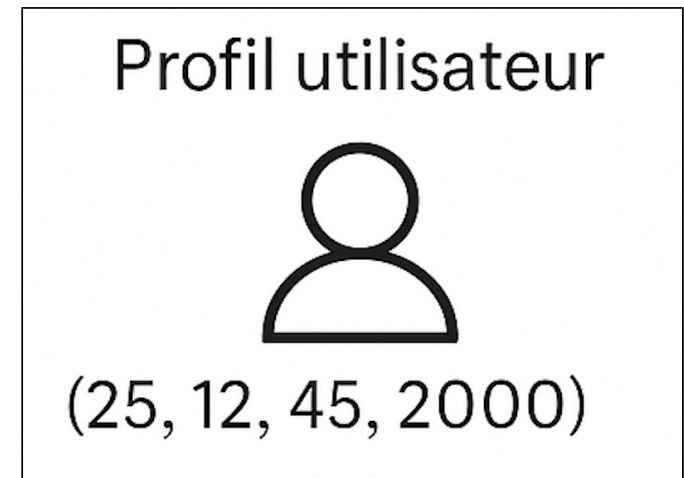
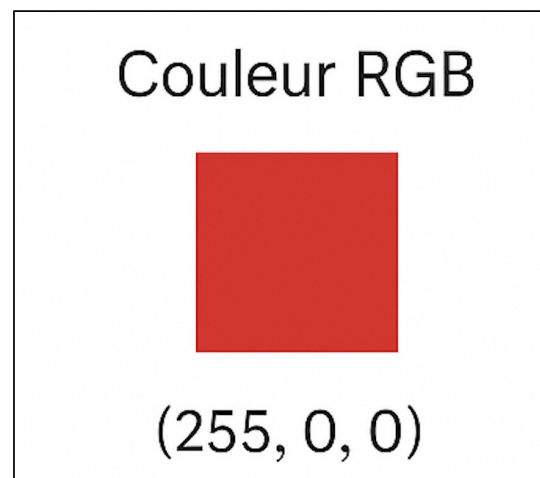
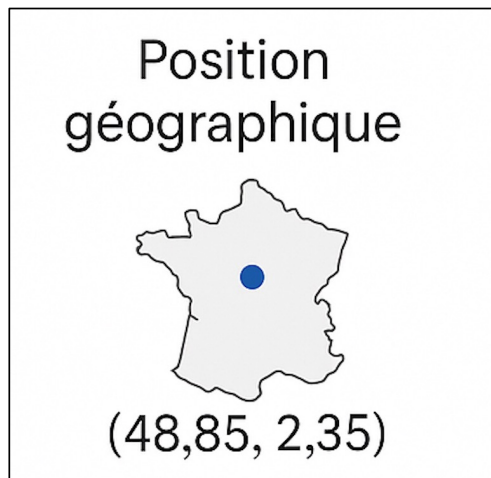
- Algorithmes de ML/DL = entrées numériques
- Sans vecteurs $\rightarrow$ pas de descente de gradient

1 – Bases mathématiques



Qu'est ce qu'un vecteur ?

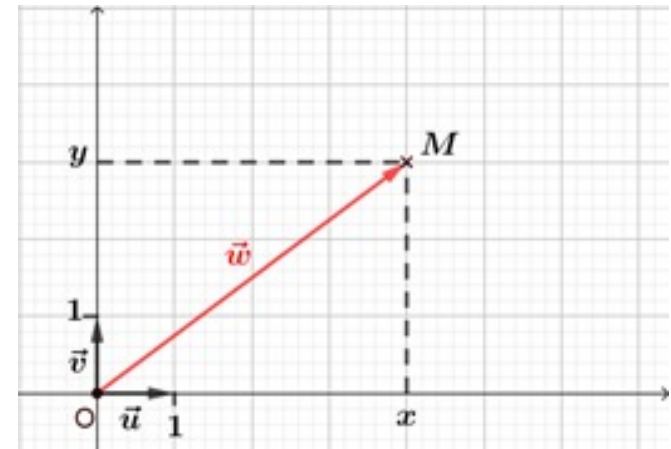
- Liste ordonnée de nombres.
- Sert à représenter des objets dans un espace mathématique.
- Exemples



Notion de vecteur en algèbre linéaire

- Suite ordonnée de nombres réels :
$$v = (x_1, x_2, \dots, x_n)$$
- En algèbre linéaire, on identifie ce vecteur avec la flèche qui va de l'origine jusqu'au point $(x_1, x_2, \dots, x_n)$
- Norme d'un vecteur : mesure de la longueur

$$\|v\| = \sqrt{x_1^2 + x_2^2 + \dots + x_n^2}$$



Coordonnées de $w = (x, y)$

$w(4, 3)$

$$\|w\| = \sqrt{4^2 + 3^2} = \sqrt{25} = 5$$

L'espace vectoriel :

le cadre mathématique pour manipuler les vecteurs

- Ensemble de vecteurs muni de deux opérations :
 - Addition de vecteurs
 - Multiplication par un scalaire
- Opérations respectant des règles (fermeture, associativité, commutativité, distributivité)
- Exemple : $\mathbb{R}^2$, $\mathbb{R}^3$, ou plus généralement $\mathbb{R}^n$.
- La dimension d'un espace vectoriel est le nombre de vecteurs de base nécessaires pour générer tous les autres.

Opérations sur les vecteurs

Soient $v = (v_1, v_2)$ et $w = (w_1, w_2)$

- addition de vecteurs

$$v + w = (v_1 + w_1, v_2 + w_2)$$

- soustraction de vecteurs

$$v - w = (v_1 - w_1, v_2 - w_2)$$

- multiplication par un scalaire

$$\text{si } \lambda \in \mathbb{R}, \lambda \cdot v = (\lambda v_1, \lambda v_2)$$

- **Produit scalaire** de v et w

$$v \cdot w = \sum v_i w_i$$

- combinaison linéaire

$$\text{Si } \alpha, \beta \in \mathbb{R}, u = \alpha v + \beta w$$

- Tout vecteur peut s'exprimer comme une combinaison linéaire d'une **base** de l'espace vectoriel

- Exemple dans $\mathbb{R}^2$

Base canonique $e_1=(1,0), e_2=(0,1)$

Tout vecteur $v = (x, y)$ peut s'écrire :

$$v = x \cdot e_1 + y \cdot e_2$$

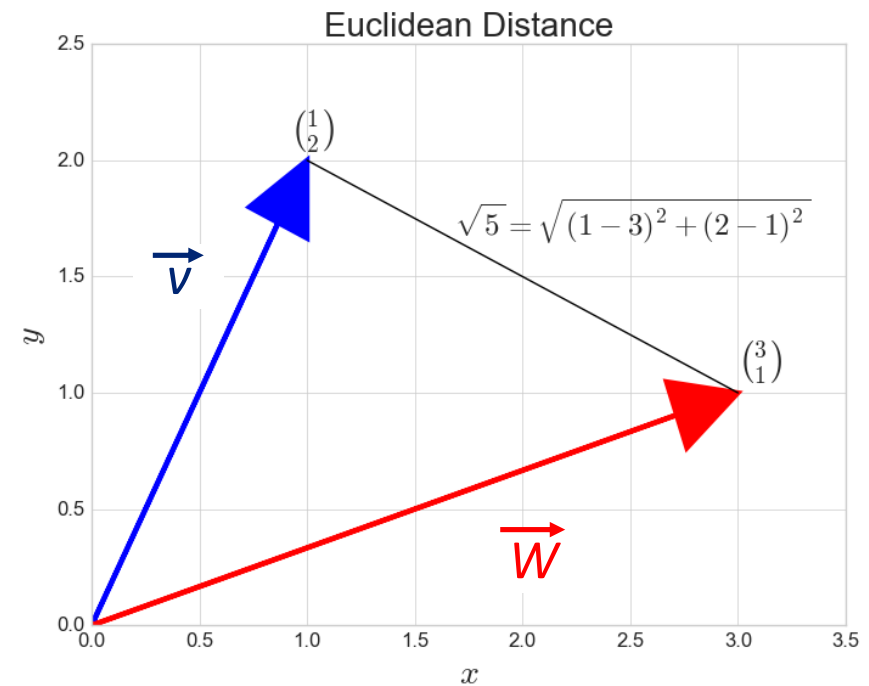
Une **base** est un ensemble minimal de vecteurs indépendants qui permet de reconstruire tout vecteur de l'espace par combinaison linéaire

Distance euclidienne entre deux vecteurs

- Mesure la proximité entre deux vecteurs comme une distance géométrique.

- $d(v,w)$
$$= \sqrt{(v_1 - w_1)^2 + (v_2 - w_2)^2 + \dots + (v_n - w_n)^2}$$

👉 Plus la distance est petite → plus les vecteurs sont proches



source : https://www.codearmo.com/media/uploads/2020/10/30/euclidean_distance_between_vectors_image.png

Pourquoi la distance euclidienne n'est pas adaptée à la similarité sémantique ?

- $u=(1,1)$ $v=(5,5)$

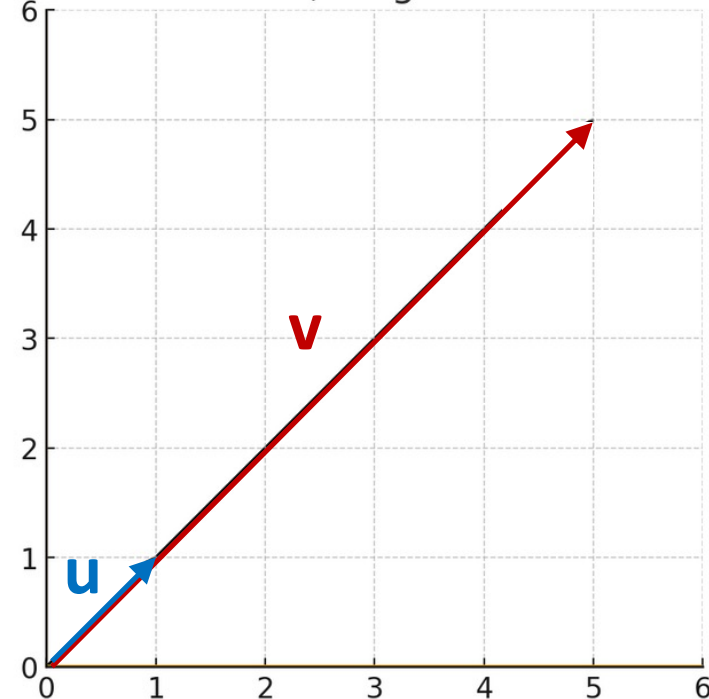
$$\|u-v\|=\sqrt{(5-1)^2+(5-1)^2}=\sqrt{32}\approx 5.657$$

- Deux vecteurs **parfaitement alignés** (même direction) peuvent avoir une distance euclidienne **grande** simplement parce que l'un est plus long !



La distance euclidienne est sensible à la longueur, pas seulement à la direction

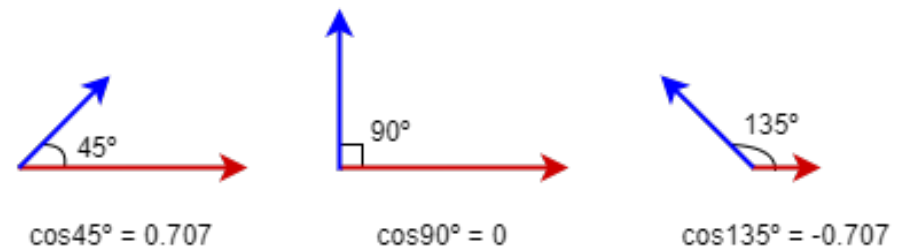
Deux vecteurs : (1,1) et (5,5)
même direction, longueurs différentes



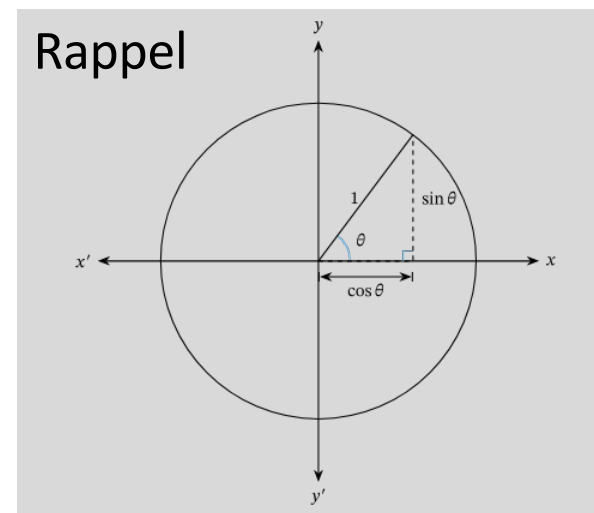
Similarité cosinus de deux vecteurs : plus adaptée à la mesure de la similarité sémantique

- La similarité cosinus est le cosinus de l'angle θ entre deux vecteurs
- Elle permet de comparer l'orientation entre deux vecteurs
 - $\text{sim}(v, w) = \frac{v \cdot w}{\|v\| \|w\|} = \cos(\theta)$
- Valeurs entre -1 et 1 :
 - proche de 1 → vecteurs très similaires (même direction)
 - proche de 0 → indépendants
 - proche de -1 → opposés

👉 Plus l'angle est petit, plus les représentants des objets proches (ou similaires)



source : <https://www.ultipa.com/fr/docs/graph-analytics-algorithms/cosine-similarity>



source : <https://www.nagwa.com/fr/explainers/825143870519/>

La similarité cosinus est focalisée sur le sens des vecteurs

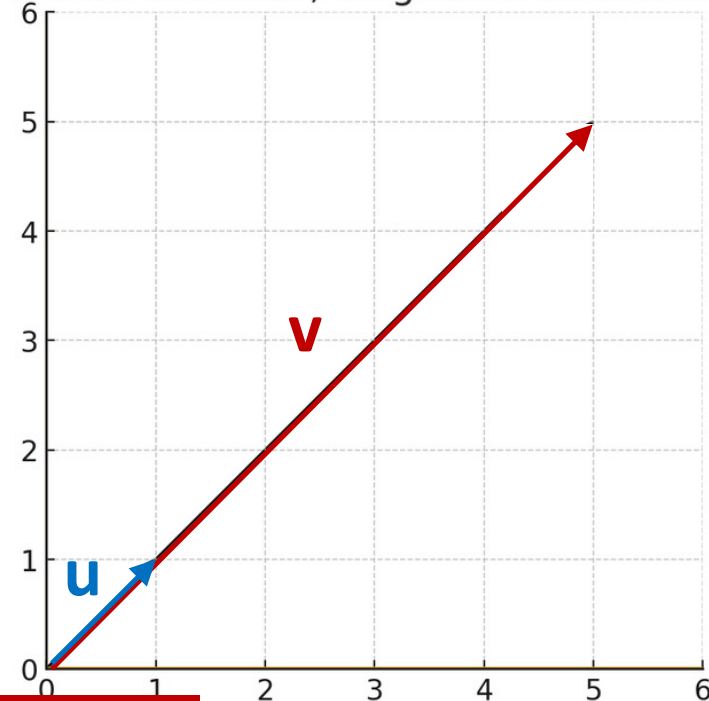
- $u=(1,1)$ $v=(5,5)$

$$\text{sim}(u, v) = \cos(u,v) = 1$$

$$\begin{aligned}\cos(u, v) &= \frac{u \cdot v}{\|u\| \|v\|} = \frac{1 \cdot 5 + 1 \cdot 5}{\sqrt{1^2 + 1^2} \sqrt{5^2 + 5^2}} \\ &= \frac{10}{\sqrt{2} \sqrt{50}} = \frac{10}{\sqrt{2} 5\sqrt{2}} = \frac{10}{10} = 1\end{aligned}$$

- La similarité représente bien le fait que u et v sont parfaitement alignés

Deux vecteurs : (1,1) et (5,5)
même direction, longueurs différentes



La similarité cosinus est particulièrement adaptée à la recherche de similarités sémantiques

Normalisation des vecteurs

- Vecteur normalisé =

Vecteur de longueur 1 obtenu en divisant chaque coordonnée par la norme du vecteur initial

$$v=[2,-1,3]$$

$$\|v\| = \sqrt{2^2 + (-1)^2 + 3^2} \approx 3,74$$

$$\hat{v} = \frac{v}{\|v\|} = \left[\frac{2}{3,74}, \frac{-1}{3,74}, \frac{3}{3,74} \right] \approx [0.53, -0.27, 0.80]$$

Très utile pour la recherche sémantique !

- Pourquoi normaliser les vecteurs?

- Rendre la distance euclidienne sensible au sens, pas à la taille
- Simplifier le calcul de la similarité cosinus

- si u et v sont normalisés alors

$$\text{sim}(v, w) = \cos(\theta) = u \cdot v$$

Orientation des vecteurs	Cosinus	Distance
Même direction	1	0
90°	0	$\sqrt{2}$
Opposés	-1	2

Vecteurs normalisés et similarités

- vecteurs normalisés

$$\hat{u} = \frac{u}{\|u\|} = \frac{(1,1)}{\sqrt{2}} = \left(\frac{1}{\sqrt{2}}, \frac{1}{\sqrt{2}}\right)$$

$$\hat{v} = \frac{v}{\|v\|} = \frac{(5,5)}{5\sqrt{2}} = \left(\frac{1}{\sqrt{2}}, \frac{1}{\sqrt{2}}\right)$$

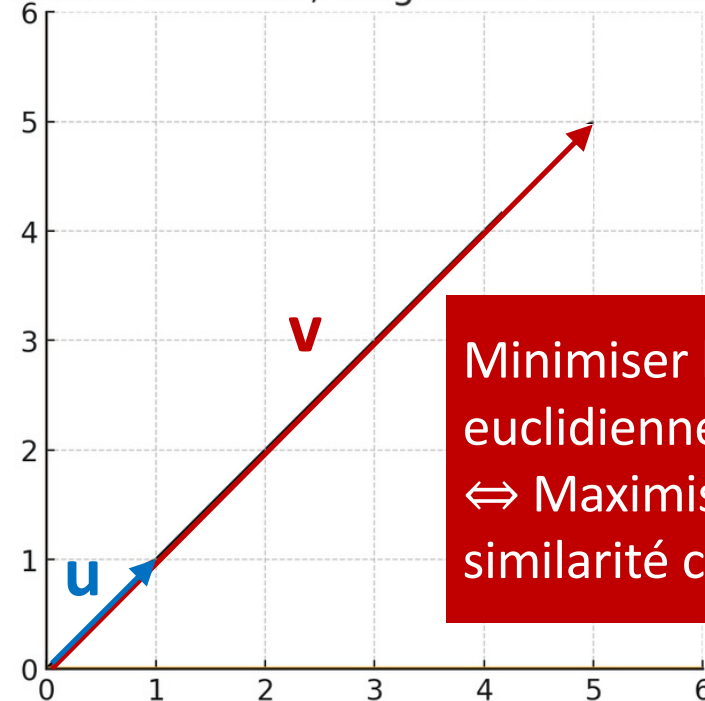
- Distance euclidienne

$$\|\hat{u} - \hat{v}\| = \|0\| = 0$$

- Similarité cosinus (pas de changement!)

- $\cos(\hat{u}, \hat{v}) = \hat{u} \cdot \hat{v} = 1$

Deux vecteurs : (1,1) et (5,5)
même direction, longueurs différentes



Minimiser la distance euclidienne
 $\Leftrightarrow$ Maximiser la similarité cosinus

Vecteurs normalisés et similarités

■ Sans normalisation

- La distance euclidienne dépend
→ **de la direction**
→ **et de la norme**
- Un vecteur plus long paraît plus éloigné même s'il pointe dans le même sens

👉 Pas adapté pour comparer des significations avec la distance euclidienne

■ Avec normalisation

- Tous les vecteurs ont la même norme (1)
- La similarité cosinus est égale au produit scalaire
- 👉 La distance euclidienne et la similarité cosinus sont reliées par la formule

$$\|\hat{u} - \hat{v}\|^2 = 2(1 - \cos(\theta))$$

Minimiser la distance euclidienne
⇔ Maximiser la similarité cosinus

Notion de distance cosinus

- Transforme la similarité cosinus en distance (mesure d'éloignement) :

- $d_{\cos}(u,v)=1-\cos(u,v)$

- Valeurs :

- $0 \rightarrow$ vecteurs identiques
 - $1 \rightarrow$ vecteurs orthogonaux
 - $2 \rightarrow$ vecteurs opposés

- **Plus la valeur est proche de 0, plus les vecteurs sont proches**

- ex: vecteurs identiques : similarité = 1 et distance cosinus=0



Pourquoi les deux existent ?

- **Similarité cosinus** (entre -1 et 1) dit *“à quel point c’est proche”*
- **Distance cosinus** (entre 0 et 2) dit *“à quel point c’est loin”*

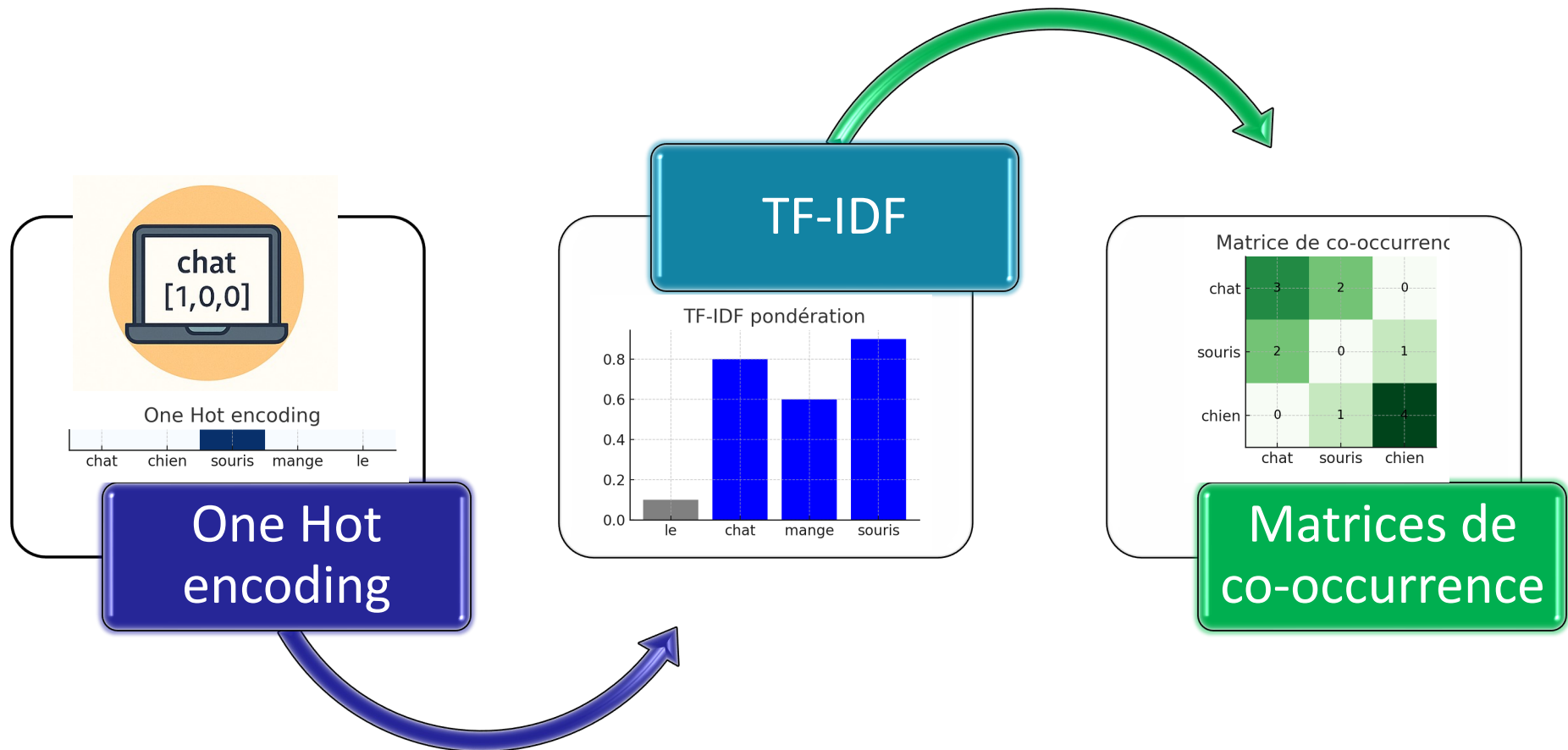
Pourquoi utiliser dans certains cas la distance euclidienne ?

- Sur des vecteurs normalisés uniquement!
- Compatibilité avec de nombreux algorithmes et outils
 - Beaucoup d'algorithmes d'indexation ont historiquement été optimisés pour la distance euclidienne
 - La plupart des algorithmes de clusterisation k-means classiques minimisent la distance euclidienne
 - configuration par défaut dans beaucoup de systèmes de recherche vectorielle
- Performance computationnelle
 - calculer une distance peut être plus rapide dans certains moteurs d'indexation

2 – Représentations vectorielles simples : symboliques et statistiques



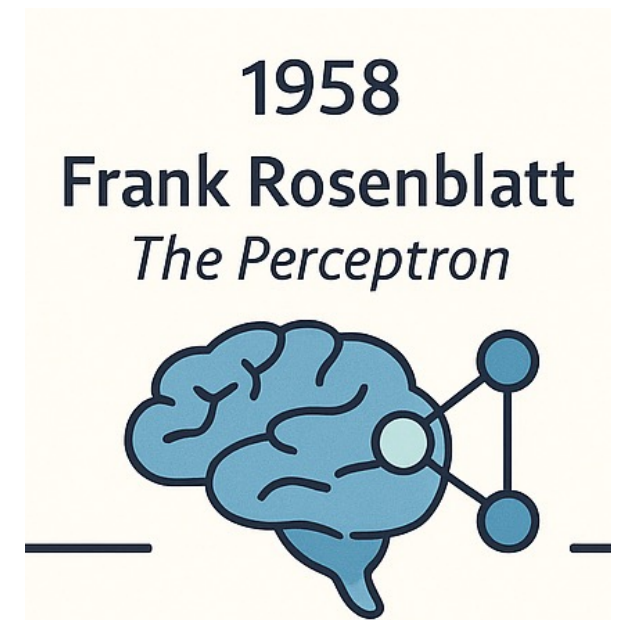
Représentations vectorielles simples



One Hot encoding

Première approche de représentation vectorielle

- Idée de représentation des entrées dans un réseau de neurone artificiel par des vecteurs binaires
- 1960-1970 - Adoption de cette approche en NLP comme codage standard des mots
 - Un mot= un vecteur unique



Vecteurs One-Hot

- Chaque symbole est représenté par un vecteur binaire de la taille du vocabulaire
- Représentation symbolique **explicite** et interprétable :
 - chaque dimension = un mot
- Vecteur **creux** (*sparse*), surtout si le vocabulaire est grand

Exemple : vocabulaire =
{chat, chien, maison, arbre}

- chat = (1, 0, 0, 0)
- chien = (0, 1, 0, 0)
- maison = (0, 0, 1, 0)
- arbre = (0, 0, 0, 1)



Espace énorme (taille du vocabulaire !!)

Vecteurs One-Hot : comparaisons ? calculs de similarité ?

- Calculs de similarité cosinus

$$\text{sim}(v, w) = \frac{v \cdot w}{\|v\| \|w\|}$$

m1 («chat») = (1, 0, 0, 0)
m2 («chien») = (0, 1, 0, 0)
m3 («maison») = (0, 0, 1, 0)
m4 («arbre») = (0, 0, 0, 1)

- Similarité m1(chat) – m2(chien)

$$m1 \times m2 = 1 \times 0 + 0 \times 1 + 0 \times 0 + 0 \times 0 = 0$$

$$\|m1\| = \|m2\| = \sqrt{1^2 + 0 + \dots + 0} = 1$$

$$\text{sim}(m1, m2) = 0$$

Aucune similarité

- Autres similarités

$$\text{sim}(m1, m3) \dots = 0$$



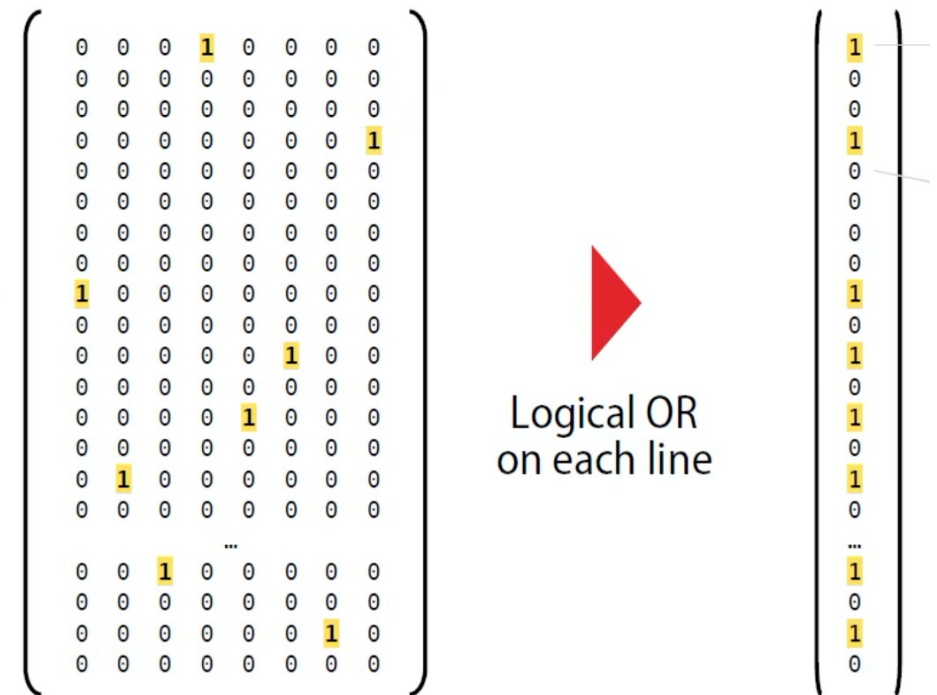
Aucune notion de sémantique

Aucune similarité identifiée:

"chat" est aussi éloigné de "chien" que de "maison"

Amélioration : One Hot Document Vector

- Un vecteur par document avec 1 pour chaque mot du vocabulaire apparaissant dans le document
- Avantages :
 - Diminution de la taille totale : un vecteur
 - Perte de l'ordre et de la fréquence des mots dans le document



source: video CNRS formation FIDLE
<https://www.youtube.com/watch?v=iUVBpTeemjU&list=PLII0-qAzf2SZHbTDoU4kFU91wqqe8N-Ug&index=2>

Bag of Words (BoW) : Term-Frequency TF

- Représenter un document comme un sac (*bag*) de mots
 - Vecteur de dimension V (taille du vocabulaire)
 - Chaque case indique la **le nombre d'occurrences** d'un mot dans le document
 - L'ordre des mots est perdu → représentation “non structurée”
- Vocabulaire = {chat, chien, maison}
 - Doc 1 : “le chat dort” → [1, 0, 0]
 - Doc 2 : “un chien chien dans la maison” → [0, 2, 1]

	Document 1	Document 2	Document 3	Document 4	Document 5	Document 6	Document 7	Document 8
Term(s) 1	10	0	1	0	0	0	0	2
Term(s) 2	0	2	0	0	0	18	0	2
Term(s) 3	0	0	0	0	0	0	0	2
Term(s) 4	6	0	0	4	6	0	0	0
Term(s) 5	0	0	0	0	0	0	0	2
Term(s) 6	0	0	1	0	0	1	0	0
Term(s) 7	0	1	8	0	0	0	0	0
Term(s) 8	0	0	0	0	0	3	0	0

Document Vector

	Doc 1	Doc 2
chat	1	0
chien	0	2
maison	0	1

Word Vector (Passage Vector)

One-hot - Bag of Words (BoW) (Term frequency (TF))

Avantages

- simplicité
- *base pour des méthodes statistiques (Naive Bayes, SVM)*

Limites

- Très grande dimension des vecteurs
- Vecteurs très creux
- Pas de prise en compte de la sémantique
- Pas de prise en compte de l'ordre des mots

Vecteurs pondérés

TF-IDF : Term Frequency – Inverse Document Frequency



Karen Spärck Jones
propose l'IDF

1972

Sparck Jones, K. (1972). A statistical interpretation of term specificity and its application in retrieval. *Journal of Documentation*, 28(1).

Salton, G., & Buckley, C. (1988). *Term-weighting approaches in automatic text retrieval*. *Information Processing & Management*, 24(5)

Salton & Buckley
popularisent **TF × IDF** comme
standard pour la recherche
d'information

1988



Gerard Salton
formalise le modèle
d'espace vectoriel

1975

[G. Salton, A. Wong, C. S. Yang, A vector space model for automatic indexing](#), *Communications of the ACM*, v.18 n.11, p. 613-620, novembre 1975

TF-IDF dans
*Introduction à la
recherche
d'information*

2008

2000s

TF-IDF devient l'outil de
référence dans la
recherche documentaire et
le TAL statistique

Manning, C. D., Raghavan, P., & Schütze, H. (2009). *Introduction to Information Retrieval*. Cambridge University Press

Vecteurs pondérés (TF-IDF) de documents

N = nb de documents

Étape 1 — Calcul de $TF(t, d)$ (term frequency)

- **$TF(t, d)$** (*term frequency*)
= fréquence du terme t dans un document d
$$= \frac{\text{nombre d'occurrences de } t \text{ dans } d}{\text{nombre total de mots dans } d}$$

Étape 2 — Calcul de $IDF(t)$ (inverse document frequency)

- **$DF(t)$** *document frequency* = nombre de documents contenant le terme t (mot comptés une seule fois)
- **$IDF(t)$** (*inverse document frequency*)
$$= \ln \left(\frac{N}{df(t)} \right)$$

$\ln$ = logarithme népérien (base e)

Étape 3 — Pondération finale : $tfidf(t, d) = tf(t, d) \times idf(t)$

Étape 4 - Vecteur TF-IDF du document

Construction d'un vecteur TF-IDF par document

Pour un vocabulaire $V = \{t_1, t_2, \dots, t_n\}$,
le document d est représenté par le vecteur :

$$d = [tfidf(t_1, d), tfidf(t_2, d), \dots, tfidf(t_n, d)]$$

Ajout d'une pondération selon
l'importance des mots dans
chaque document

Vecteurs pondérés (TF-IDF) de documents

Exemple

Exemple : Corpus de 3 phrases

d1 : « chat dort »

d2 : « chien court »

d3 : « maison dort »

Vocabulaire : {chat, chien, maison, dort, court}

$N = 3$

Étape 1 — TF(t, d) - term frequency

Mot	tf(t,d ₁)	tf(t,d ₂)	tf(t,d ₃)
chat	0.5 = 1/2	0	0
chien	0	0.5	0
maison	0	0	0.5
dort	0.5=1/2	0	0.5
court	0	0.5	0

Étape 2 — Calcul de IDF(t) - inverse document frequency

df(t) = nombre de documents contenant t

$$\text{IDF}(t) = \ln \left(\frac{N}{df(t)} \right)$$

Mot	df(t)	IDF(t)
chat	1	$\ln(3/1) = \mathbf{1.10}$
chien	1	1.10
maison	1	1.10
dort	2	$\ln(3/2) = \mathbf{0.41}$
court	1	1.10

- Les mots rares (chat chien, maison, court) ont un idf élevé 1.10 : ils caractérisent fortement leur document
- dort est plus fréquent (idf 0,41): il est moins discriminant

Vecteurs pondérés (TF-IDF) de documents

Exemple



Étape 3 — Pondération finale

$$TFIDF(t,d)=TF(t,d)\times IDF(t)$$

Mot	$tfidf(t,d_1)$	$tfidf(t,d_2)$	$tfidf(t,d_3)$
chat	$0.5 \times 1.10 = \mathbf{0.55}$	0	0
chien	0	$0.5 \times 1.10 = \mathbf{0.55}$	0
maison	0	0	$0.5 \times 1.10 = \mathbf{0.55}$
dort	$0.5 \times 0.41 = \mathbf{0.205}$	0	$0.5 \times 0.41 = \mathbf{0.205}$
court	0	$0.5 \times 1.10 = \mathbf{0.55}$	0



Etape 4 - Vecteur TF-IDF du document

Document	Vecteur TF-IDF = (chat, chien, maison, dort, court)
d₁	(0.55, 0, 0, 0.205 , 0)
d₂	(0, 0.55, 0, 0, 0.55)
d₃	(0, 0, 0.55, 0.205 , 0)

Vecteurs pondérés (TF-IDF)



Avantage

- Calculs de similarité plus significatifs
- Ebauche de sémantique*

- Calculs de similarité cosinus entre documents

$$\text{sim}(v, w) = \frac{v \cdot w}{\|v\| \|w\|}$$

d1 (« chat dort ») = (0.55, 0, 0, **0.205**, 0)
d2 (« chien court ») = (0, 0.55, 0, 0, 0.55)
d3 (« maison dort ») = (0, 0, 0.55, **0.205**, 0)



Limites

- Taille toujours énorme
- Vecteurs creux
- Pas de synonymie
- *Pas de contexte*

1. Normes des vecteurs

- $\|d1\| = \sqrt{0.55^2 + 0.205^2} \approx 0.587$
- $\|d2\| = \sqrt{0.55^2 + 0.55^2} \approx 0.778$
- $\|d3\| = \sqrt{0.55^2 + 0.205^2} \approx 0.587$

2. Similarité d1 – d2

$$d1.d2 = \text{sim}(d1, d2) = 0$$

Aucune similarité

3. Similarité d1 – d3

$$d1.d3 = 0.205 \times 0.205 = 0.0420$$

$$\text{sim}(d1, d3) = d1.d3 / (0.587 \times 0.587) \approx 0.344$$

Partage du mot "dort"

4. Similarité d2 – d3

$$d2.d3 = \text{sim}(d2, d3) = 0$$

Aucune similarité

A vous de jouer : Exercice (1/3)



Corpus (Nombre de documents : $N = 3$)

d1 : « soleil brille fort »

d2 : « lune brille doucement »

d3 : « maison ancienne belle »

Vocabulaire et ordre figé (dimension = 8) :

soleil, lune, brille, fort, doucement, maison, ancienne, belle

Q1 — Représentation one-hot

- Donnez les vecteurs one-hot des mots « soleil », « lune », « maison » dans l'ordre du vocabulaire.
- Calculez la similarité cosinus entre les paires (soleil, lune), (soleil, maison) et (lune, maison) (*expliquez vos calculs*).
- Pourquoi cette mesure n'apporte-t-elle aucune information sémantique avec le one-hot ?

A vous de jouer : Exercice (1/3)

correction



Q1 — Représentation one-hot

- vecteurs one-hot

soleil = (1,0,0,0,0,0,0,0)

lune = (0,1,0,0,0,0,0,0)

maison = (0,0,0,0,0,1,0,0)

- Calculs similarité cosinus

$\text{sim}(\text{soleil}, \text{lune}) = 0$

$\text{sim}(\text{soleil}, \text{maison}) = 0$

$\text{sim}(\text{lune}, \text{maison}) = 0$

Conclusion : en one-hot, des mots différents sont orthogonaux ($\cos=0$) → aucune proximité sémantique capturée.

A vous de jouer : Exercice (2/3)



Q2 — Représentation TF-IDF (base e)

- Etapes 1 et 2 - Calculez $TF(t)$, $DF(t)$ et $IDF(t)$ pour chaque terme du vocabulaire
- Etapes 3 et 4 - Construisez les vecteurs TF-IDF complets (dimension 8) de $d1$, $d2$ et $d3$ (dans l'ordre du vocabulaire)

Q3 — Similarité cosinus

- Calculez $sim(d1,d2)$, $sim(d1,d3)$ et $sim(d2,d3)$
- Interprétez : quelle paire est la plus proche ? Est-ce cohérent avec l'intuition ?

A vous de jouer : Exercice (2/3)

correction



Q2 — Représentation TF-IDF (base e)

- TF, DF et IDF

Mot	tf(t,d ₁)	tf(t,d ₂)	tf(t,d ₃)	DF(t)	IDF(t)	TF-IDF d1	TF-IDF d2	TF-IDF d2
soleil	1/3=0.33	0	0	1	$\ln(3/1)=1.10$	$0.33 \times 1.10 = 0.37$	0	0
lune	0	1/3=0.33	0	1	1.10	0	$0.33 \times 1.10 = 0.37$	0
brille	1/3=0.33	1/3=0.33	0.5	2	$\ln(3/2)=0.41$	$0.33 \times 0.41 = 0.14$	$0.33 \times 0.41 = 0.14$	0
fort	1/3=0.33	0	0.5	1	1.10	$0.33 \times 1.10 = 0.37$	0	0
doucelement	0	1/3=0.33	0	1	1.10	0	$0.33 \times 1.10 = 0.37$	0
maison	0	0	1/3=0.33	1	1.10	0	0	$0.33 \times 1.10 = 0.37$
ancienne	0	0	1/3=0.33	1	1.10	0	0	$0.33 \times 1.10 = 0.37$
belle	0	0	1/3=0.33	1	1.10	0	0	$0.33 \times 1.10 = 0.37$

- d1=[0.37, 0, 0.14, 0.37, 0, 0, 0, 0]
- d2=[0, 0.37, 0.14, 0, 0.37, 0, 0, 0]
- d3=[0, 0, 0, 0, 0, 0.37, 0.37, 0.37]

A vous de jouer : Exercice (2/3)

correction



Q2 — Représentation TF-IDF (base e)

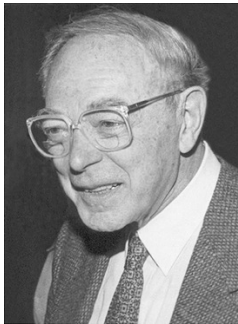
- Vecteurs TF-IDF (ordre : *soleil, lune, brille, fort, doucement, maison, ancienne, belle*) :
 - $d1 = [0.37, 0, 0.14, 0.37, 0, 0, 0, 0]$
 - $d2 = [0, 0.37, 0.14, 0, 0.37, 0, 0, 0]$
 - $d3 = [0, 0, 0, 0, 0, 0.37, 0.37, 0.37]$

Q3 — Similarité cosinus

- Calcul des normes :
- $\|d_1\| = \|d2\| = \sqrt{0.37^2 + 0.14^2 + 0.37^2} = \sqrt{0.1369 + 0.0196 + 0.1369} = \sqrt{0.2934} \approx 0.541$
- $\|d3\| \approx \sqrt{0.37^2 + 0.37^2 + 0.37^2} \approx 0.641$
- Similarités cosinus
 - $d1 \cdot d2 = 0.14 \times 0.14 \approx 0.0196 \Rightarrow \text{sim}(d1, d2) \approx \frac{0.0196}{0.541 \times 0.541} \approx 0,067$
 - $d1 \cdot d3 = 0$ (aucun terme commun) $\Rightarrow \text{sim}(d1, d3) = 0$
 - $d2 \cdot d3 = 0 \Rightarrow \text{sim}(d2, d3) = 0$
- Interprétation : $d1$ et $d2$ sont les plus proches (partagent « brille ») (similarité faible). $d3$ est orthogonal aux deux autres. Cohérent avec l'intuition.

Matrices de co-occurrence : La première mise en pratique de l'hypothèse distributionnelle

- Le sens d'un mot est défini par les contextes dans lesquels il apparaît

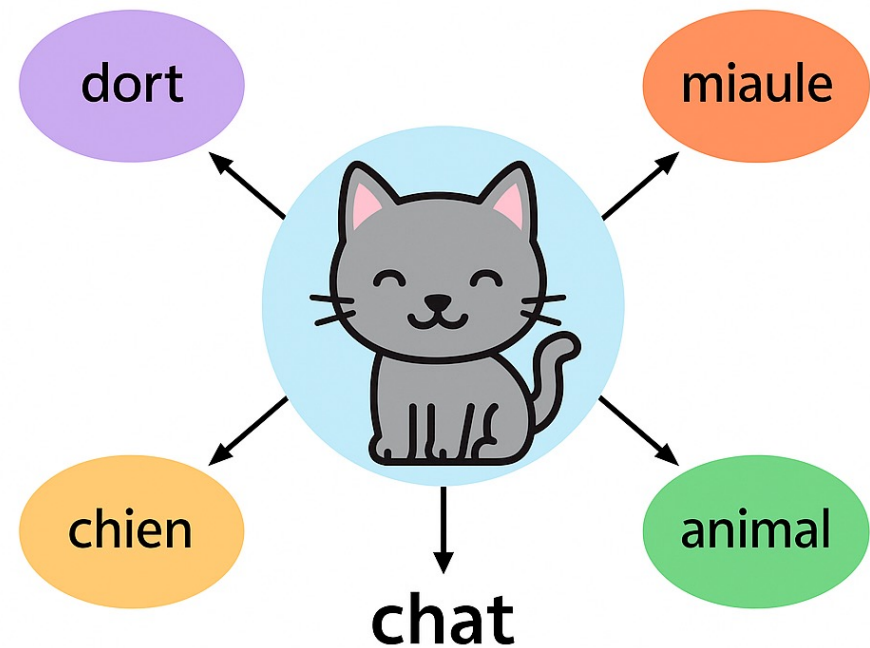


- "The meaning of entities is related to the distributions of the forms in which they appear."

Harris, Z. (1954). *Distributional structure*. *Word*, 10(2/3), 146–162.

<https://www.tandfonline.com/doi/pdf/10.1080/00437956.1954.11659520>

- Utilisées dès les **années 1960–70** pour analyser la proximité lexicale



Hypothèse fondamentale utilisée par les LLM

Qu'est-ce qu'une matrice de co-occurrence ?

- Chaque mot est représenté par son **vecteur de co-occurrence** (la ligne correspondante)
 - Deux mots apparaissant dans des contextes similaires → vecteurs proches
- Colonnes = mots (ou contextes)
- Cellule $M_{i,j}$ = nombre de fois où le mot i apparaît dans le contexte du mot j
- Le contexte peut être :
 - une fenêtre de quelques mots autour,
 - ou bien la phrase/document

mot \ mot	mot 1	mot 2	...	mot n
mot 1	0	2	0	1
mot 2	2	0	0	1
...	0	0	0	0
mot n	1	1	0	0

le mot 1 apparaît 2 fois dans le contexte du mot 2 et 1 fois dans le contexte du mot n



Limites

- Taille toujours énorme
- Vecteurs creux



Avantage

- Calculs de similarité plus significatifs

Exemple de matrice de co-occurrence

Corpus

- d1 : « Le chat mange des croquettes »
 - d2 : « Le chien dévore des os »
 - d3 : « Un chat est un animal domestique »
 - d4 : « Un chien est un animal domestique »
- **Fenêtre choisie**: un mot à gauche et à droite
 - Un mot = un vecteur (**une ligne de la matrice**)

mot\ mot	chat	chien	mange	dévore	animal	domestique	croquettes	os
chat	0	0	1	0	1	0	0	0
chien	0	0	0	1	1	0	0	0
mange	1	0	0	0	0	0	1	0
dévore	0	1	0	0	0	0	0	1
animal	1	1	0	0	0	2	0	0
domestique	0	0	0	0	2	0	0	0
croquettes	0	0	1	0	0	0	0	0
os	0	0	0	1	0	0	0	0

Exemple de matrice de co-occurrence

Calculs des similarités cosinus

- Similarité chat – chien
 - Produit scalaire $chat.chien=1$
 - Normes $||chat||=\sqrt{2}$ $||chien||=\sqrt{2}$
- $$sim(chat, chien) = \frac{1}{2} = 0,5$$

"chat" et "chien" apparaissent
comme sémantiquement proches
grâce au partage du contexte

TF-IDF n'aurait trouvé aucune
similarité: Pourquoi?

mot\ mot	chat	chien	mange	dévore	animal	domestique	croquettes	os
chat	0	0	1	0	1	0	0	0
chien	0	0	0	1	1	0	0	0

A vous de jouer : Exercice (3/3)



Corpus (Nombre de documents : $N = 3$)

d1 : « soleil brille fort »

d2 : « lune brille doucement »

d3 : « maison ancienne belle »

Q4 — Construire la matrice de co-occurrence des mots du vocabulaire '*soleil, lune, brille, fort, doucement, maison, ancienne, belle*' en considérant que deux mots co-apparaissent s'ils sont dans le même document (fenêtre choisie : 2 mots à gauche et à droite)

- Calculer la similarité cosinus entre les vecteurs de soleil et lune. Que montre ce calcul par rapport à TF-IDF ?

A vous de jouer : Exercice (3/3)

correction

Q4 — Matrice de co-occurrence



mot\ mot	soleil	lune	brille	fort	doucelement	maison	ancienne	belle
soleil	0	0	1	1	0	0	0	0
lune	0	0	1	0	1	0	0	0
brille	1	1	0	1	1	0	0	0
fort	1	0	1	0	0	0	0	0
doucelement	0	1	1	0	0	0	0	0
maison	0	0	0	0	0	0	1	1
ancienne	0	0	0	0	0	1	0	1
belle	0	0	0	0	0	1	1	0

A vous de jouer : Exercice (3/3)

correction



Q4 — Matrice de co-occurrence

soleil	0	0	1	1	0	0	0	0
lune	0	0	1	0	1	0	0	0

Produit scalaire :

$$\text{soleil.lune} = (0 \times 0) + (0 \times 0) + (1 \times 1) + (1 \times 0) + (0 \times 1) = 1$$

Normes :

$$\| \text{soleil} \| = \sqrt{1^2 + 1^2} = \sqrt{2}$$

$$\| \text{lune} \| = \sqrt{1^2 + 1^2} = \sqrt{2}$$

Similarité cosinus :

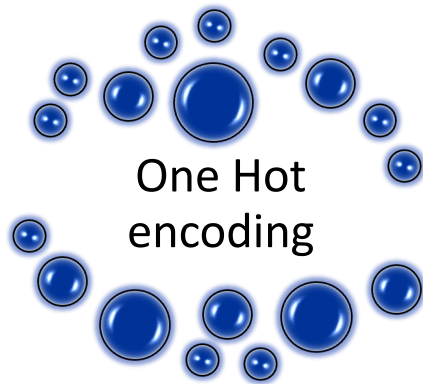
$$\text{sim}(\text{soleil}, \text{lune}) = \frac{1}{\sqrt{2} \times \sqrt{2}} = 1/2 = 0,5$$

👉 **Matrice de co-occurrence** → révèle que *soleil* et *lune* partagent un voisin commun (*brille*), d'où une similarité de 0,5.

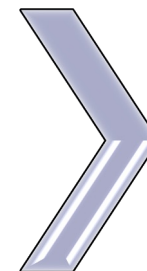
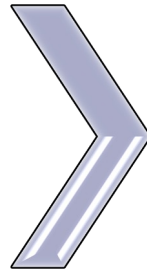
👉 **TF-IDF** → ne relie pas *soleil* et *lune*, car ils n'apparaissent pas dans le même document

👉 **Apport clé** : la co-occurrence permet de détecter des **similarités lexicales implicites** entre mots, pas seulement entre documents.

Résumé des représentations simples



One Hot
encoding



Matrices de
co-occurrence

- ✓ Simple, fidèle au symbolique
- ✗ Vecteurs très grands et creux
- ✗ Aucune similarité sémantique
- 🔍 **Explicite** : chaque dimension = un mot

- ✓ Pondération plus fine (mots rares valorisés)
- ✓ Permet de comparer documents par similarité cosinus
- ✗ Toujours des vecteurs creux
- ✗ Pas de synonymie, pas de contexte
- 🔍 **Explicite** : pondérations interprétables

- ✓ Capturent les relations entre **mots** (et non seulement entre documents)
- ✓ Première implémentation de l'**hypothèse distributionnelle**
- ✗ Toujours creuses et de très grande dimension
- 🔍 **Explicite** : interprétation possible des co-occurrences

Quand les vecteurs simples ne suffisent plus ...

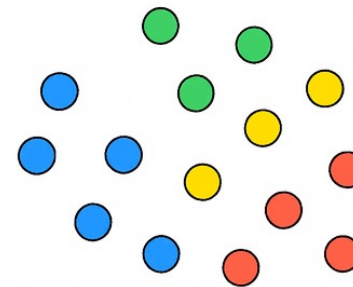
Limites des représentations simples

- ❌ Vecteurs énormes (taille = vocabulaire) → inefficaces
- ❌ Pas de vraie notion de proximité sémantique
- ❌ Difficile de comparer différents types de données

0	0	1	0	0
---	---	---	---	---

💡 Besoins des systèmes modernes (LLM, IA multimodale)

- ✅ Représentations plus compactes
- ✅ Représentations qui capturent la similarité de sens
- ✅ Un principe général et universel (texte, image, son)



3 – Vers les représentations denses : la notion d'embedding



Premières étapes vers la densité : LSA

Latent Semantic Analysis (LSA) (Deerwester et al., 1990)

- Deerwester, S. et al. (1990). *Indexing by Latent Semantic Analysis*. *Journal of the American Society for Information Science*, 41(6).
http://wordvec.colorado.edu/papers/Deerwester_1990.pdf

Principe

1. Construire une matrice terme-document (ou mot-contexte)
 - Chaque cellule contient une pondération (souvent TF-IDF)
 - Cette matrice est immense et creuse
2. Appliquer SVD Décomposition en Valeurs Singulières pour réduire la dimensionnalité et créer des vecteurs denses

Doc1 = "chat noir"
Doc2 = "chat blanc"
Doc3 = "chien noir"

Terme	Doc1	Doc2	Doc3
chat	1	1	0
chien	0	0	1
noir	1	0	1
blanc	0	1	0

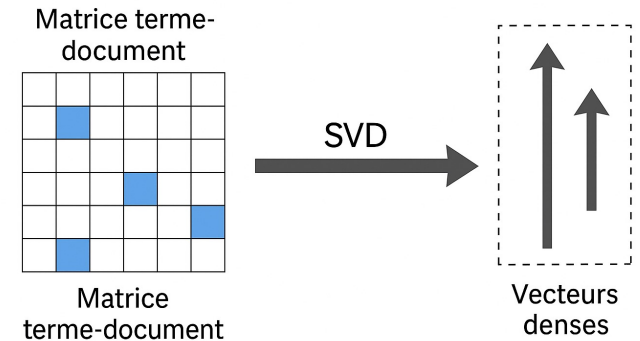
dimensions latentes

Mot	Dim1	Dim2
chat	0.9	0.1
chien	0.8	-0.2
noir	0.6	0.5
blanc	-0.5	0.6

Doc	Dim1	Dim2
Doc1	0.7	0.3
Doc2	0.2	0.6
Doc3	0.8	-0.1

Apports de LSA

- Révélation des structures latentes dans les co-occurrences
- Des mots qui n'apparaissent pas ensemble peuvent tout de même être rapprochés via les dimensions latentes



Mot	Dim1	Dim2
chat	0.9	0.1
chien	0.8	-0.2
noir	0.6	0.5
blanc	-0.5	0.6

**Dimensions "latentes" =
dimensions découvertes
automatiquement par la
décomposition SVD**

`similarite_cosinus(chat, chien) ≈ 0.94`

Limites de LSA

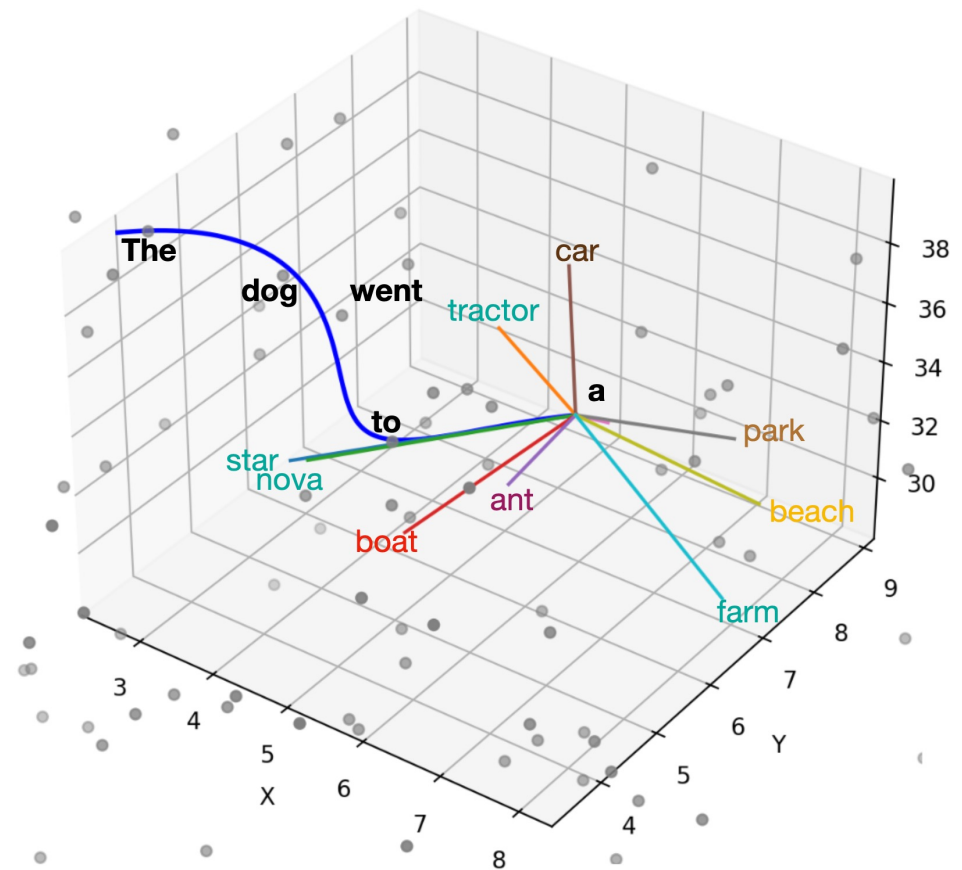
- La réduction de dimension est globale et figée → pas de mise à jour facile avec de nouvelles données
- Pas de prise en compte fine du contexte :
 - un mot = toujours le même vecteur
- Les relations captées sont assez grossières, car purement statistiques
- Lourdeur calculatoire : les matrices sont immenses et coûteuses à décomposer

LSA a ouvert la voie mais ses limites ont conduit aux embeddings

Qu'est-ce qu'un embedding ?

- Un vecteur avec des centaines voire des **milliers de dimensions** :
 - une dimension représente différents aspects sémantiques ou caractéristiques de l'entité représentée (mot, pixel, film, playlist...).
- Les dimensions n'ont **pas une interprétation humaine** directe

le nombre de dimensions d est un paramètre du modèle

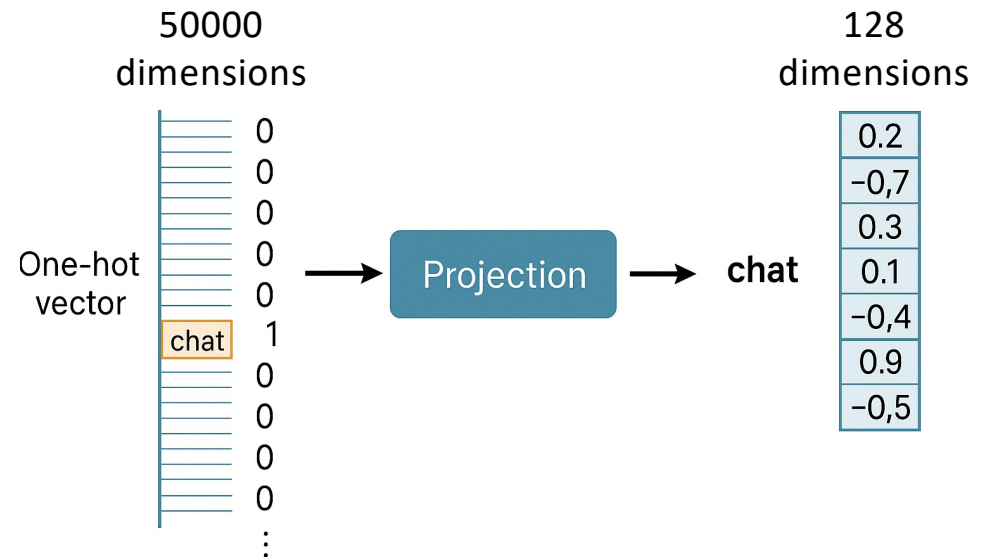


source : <https://ironj.github.io/llmviz/>

Définition d'un embedding

- Vecteur

- **dense**
- **construit par apprentissage** :
encode des proximités sémantiques
automatiquement quantifiables
- **continu** : valeurs réelles



Un embedding est une fonction $f: V \rightarrow \mathbb{R}^d$ apprise à partir des données
 V = **vocabulaire** ou ensemble d'objets à représenter (taille très grande)
 d = **dimension de l'espace projeté** : choisie par le concepteur (hyperparamètre)
(ex. 100–1024)

Embeddings et sémantique

- La **proximité géométrique** traduit une proximité sémantique.

- Conséquences

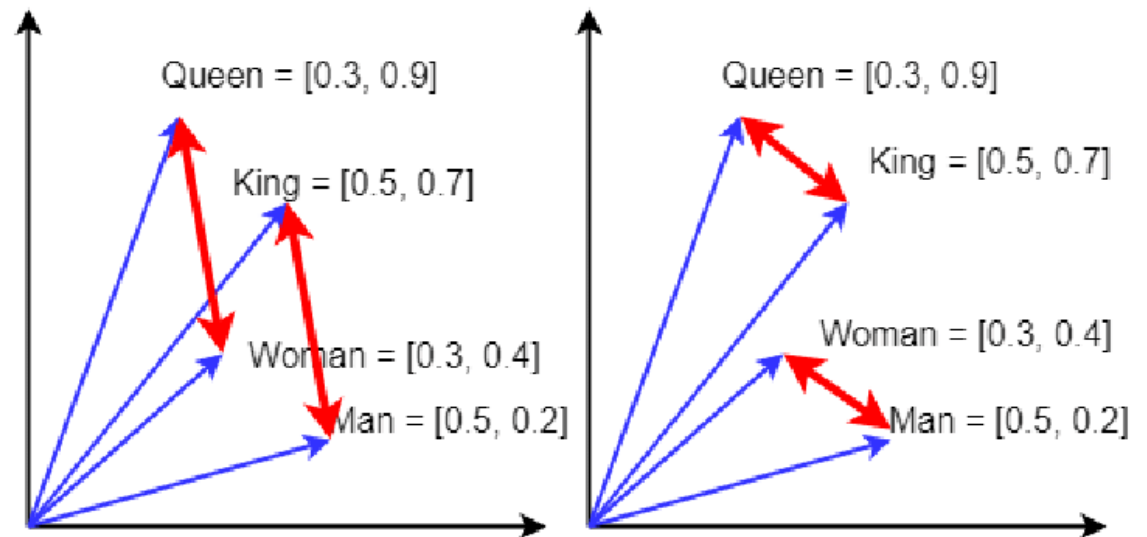
- Les relations de sens deviennent mesurables (distances, angles) :

- Calculs de similarité significatifs

- Les opérations vectorielles ont des significations :

- $King - Man + Woman \approx Queen$

$$[0.5, 0.7] - [0.5, 0.2] + [0.3, 0.4] = [0.3, 0.9] = Queen$$



Sur quoi est basée la construction des embeddings ?

Toujours l'hypothèse fondamentale de la sémantique distributionnelle

- **"You shall know a word by the company it keeps"**

(Firth, J. R. 1957. *Papers in Linguistics 1934–1951*, Oxford University Press.)



Discutable ? Qu'en pensez vous ?



Critiques de l'hypothèse distributionnelle

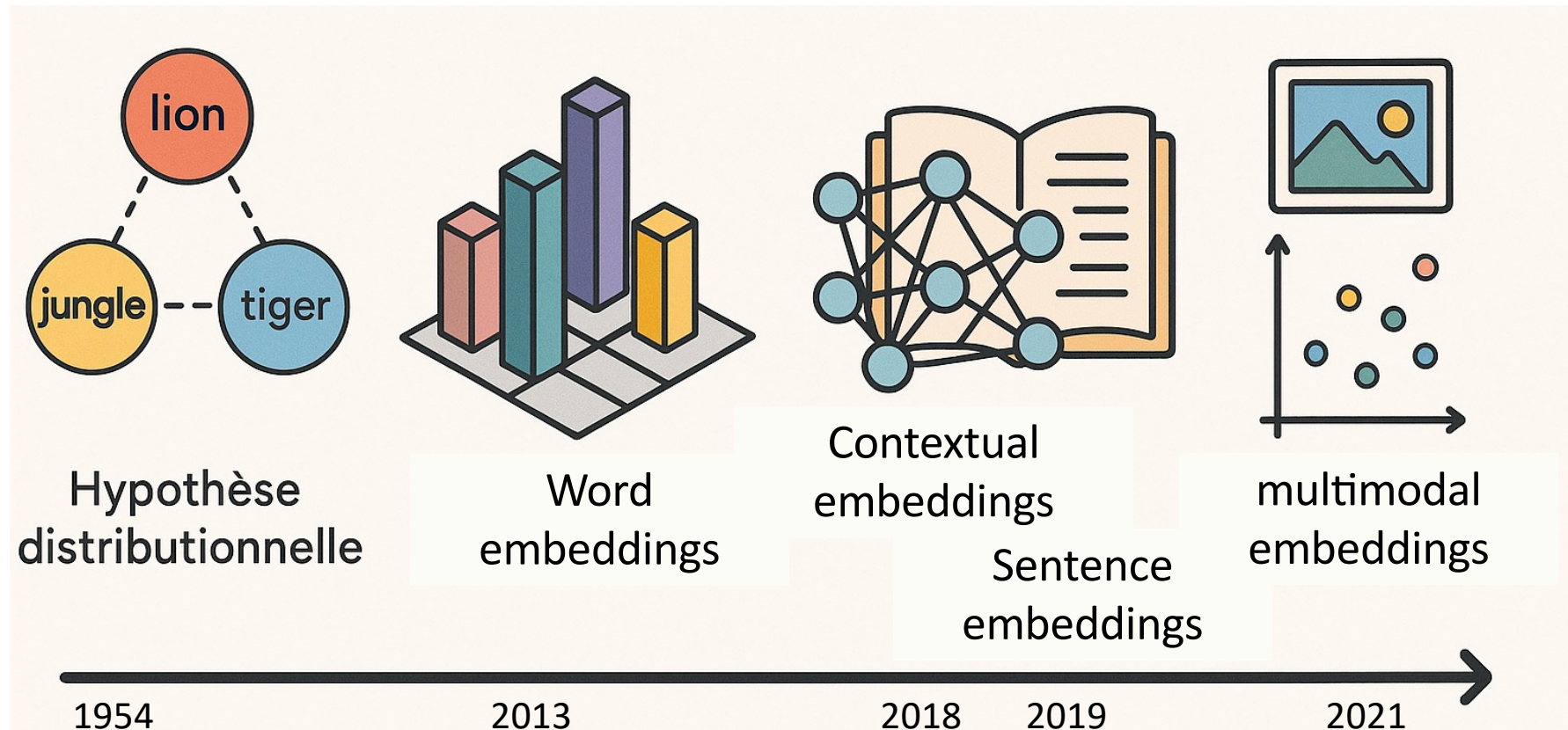
- Fréquence = vérité ??
- Représentation de la compréhension humaine ?
 - L'apprentissage humain n'est pas uniquement statistique : perception, action, émotion et capacités innées
 - Importance de l'interaction, de l'attention conjointe, de l'intersubjectivité entre humains



Emily BENDER

— Bender, E. M., & Koller, A. (2020). *Climbing towards NLU: On meaning, form, and understanding in the age of data*. ACL.
<https://aclanthology.org/2020.acl-main.463/>

Historique des embeddings



Embeddings : ce qui change selon les modèles

Modèle	Niveau de représentation	Exemple corpus jouet « La batterie du téléphone est vide. » « Le musicien joue de la batterie. »	Ce que ça change
Word2Vec / GloVe (statiques)	Un seul vecteur par mot. Word Embedding	batterie → [0.21, -0.55, 0.89]	❌ Ne distingue pas énergie vs instrument
ELMo (LSTM, semi-contextuel)	Vecteur différent selon le contexte local Contextual Embedding	batterie (téléphone) → [0.11, -0.09, 0.65] batterie (musique) → [-0.73, 0.52, -0.12]	✅ Sens différencié grâce au voisinage immédiat
BERT / GPT (transformers)	Vecteur contextualisé par toute la phrase	batterie (téléphone) → [0.02, -0.44, 0.93, ...] batterie (musique) → [-0.81, 0.18, -0.27, ...]	✅ Distinction plus fine et robuste
Sentence-BERT	Un vecteur unique par phrase entière Sentence Embedding	« La batterie du téléphone est vide. » → [0.31, -0.22, 0.77, ...] « Le musicien joue de la batterie. » → [-0.84, 0.15, -0.41, ...]	✅ Comparaison directe de phrases, ❌ perte de nuance intra-phrase

Paramètres des modèles à embeddings



Corpus d'entraînement

- Taille
- Domaine
- Qualité des données



Modèle choisi

- Statique (Word2Vec, GloVe)
- Contextuel (BERT, GPT)
- Sentence
- Multimodal (CLIP...)



Paramètres de conception

- Dimension de l'espace latent (d)
- Taille de la fenêtre de contexte
- Fonction d'objectif (Skip-gram, CBOW, masquage, etc.)



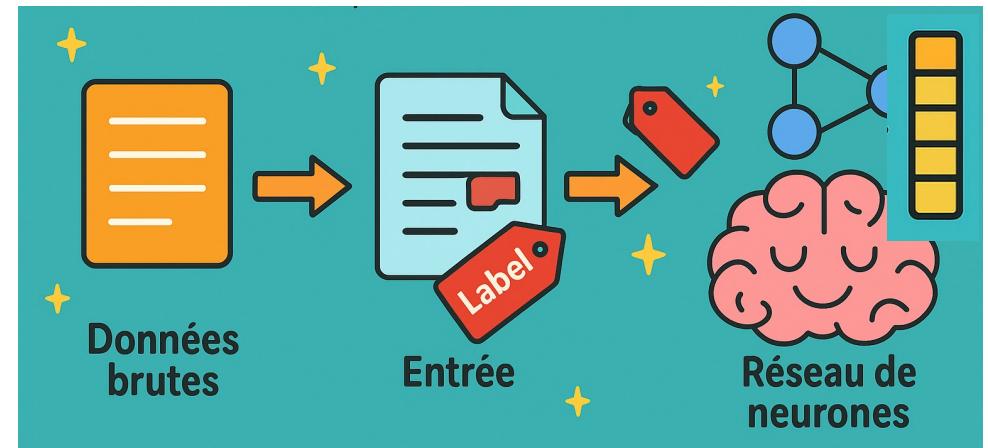
Paramètres d'entraînement (optimisation)

- Taux d'apprentissage (**learning rate**) → vitesse de mise à jour
- Nombre d'itérations (**epochs**) → combien de fois on passe sur le corpus
- Taille du **batch** → combien d'exemples pour une mise à jour
- Negative sampling → combien d'exemples négatifs générés
- Algorithme d'optimisation → SGD Stochastic Gradient Descent , Adam, ...

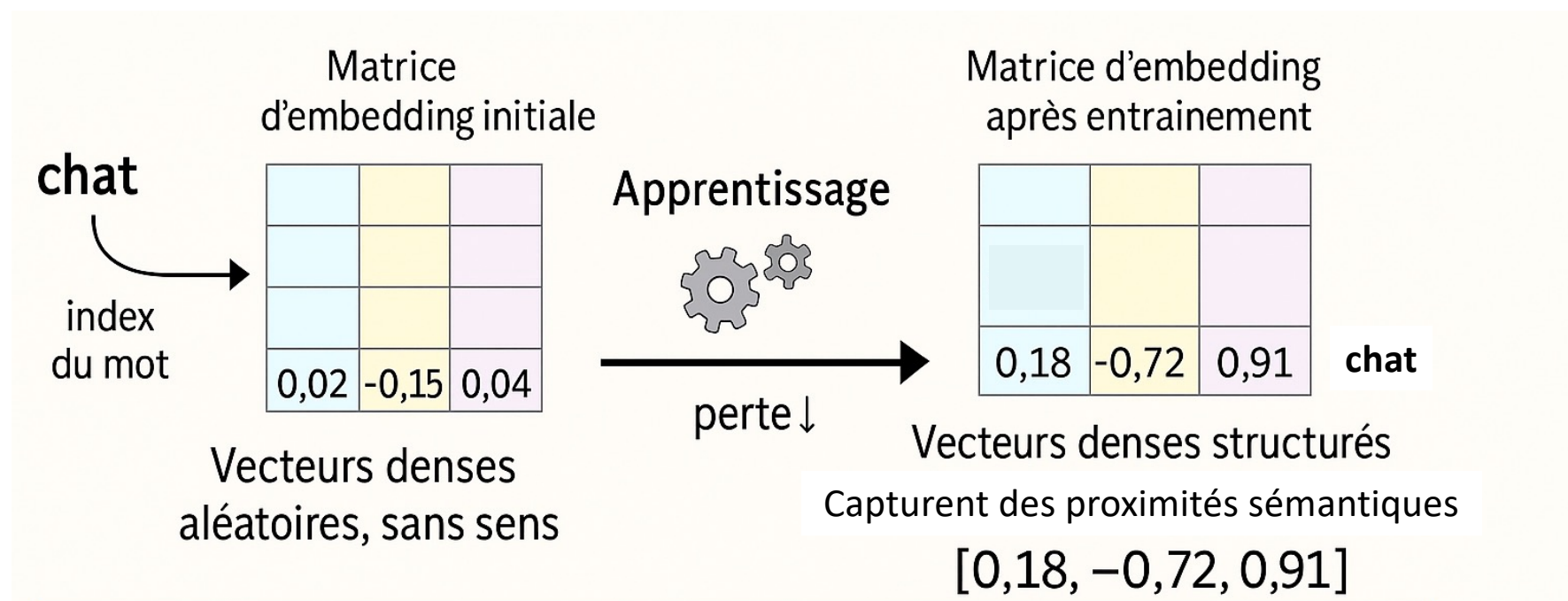
Principe d'apprentissage auto-supervisé

🗨 Le modèle fabrique lui-même sa supervision en construisant automatiquement des paires (entrée, label) à partir des données brutes

- Par exemple :
 - prédire un mot manquant,
 - prédire le mot suivant,
 - reconstituer une image dégradée, etc.
- On parle aussi de **prétexte task** (*tâche de pré-entraînement*).



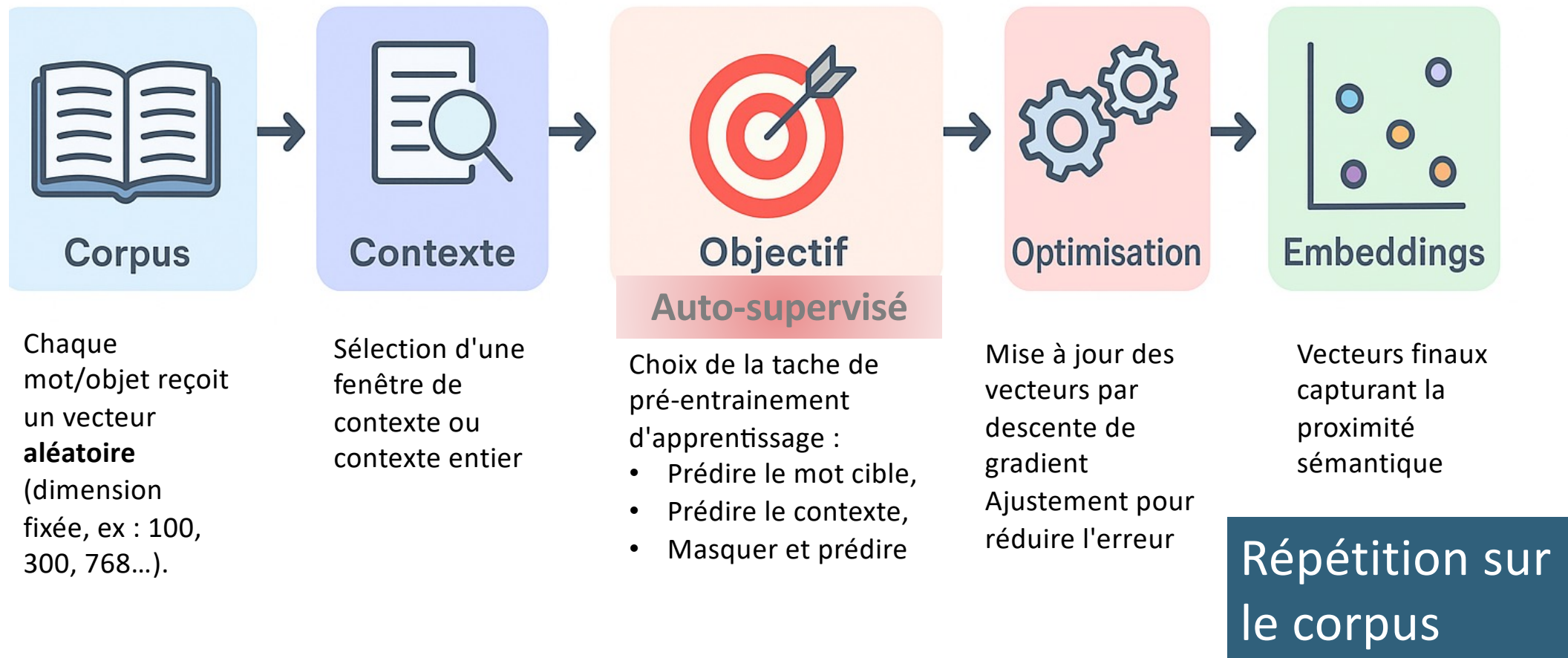
Le principe général d'apprentissage des embeddings : apprendre le sens à partir du hasard



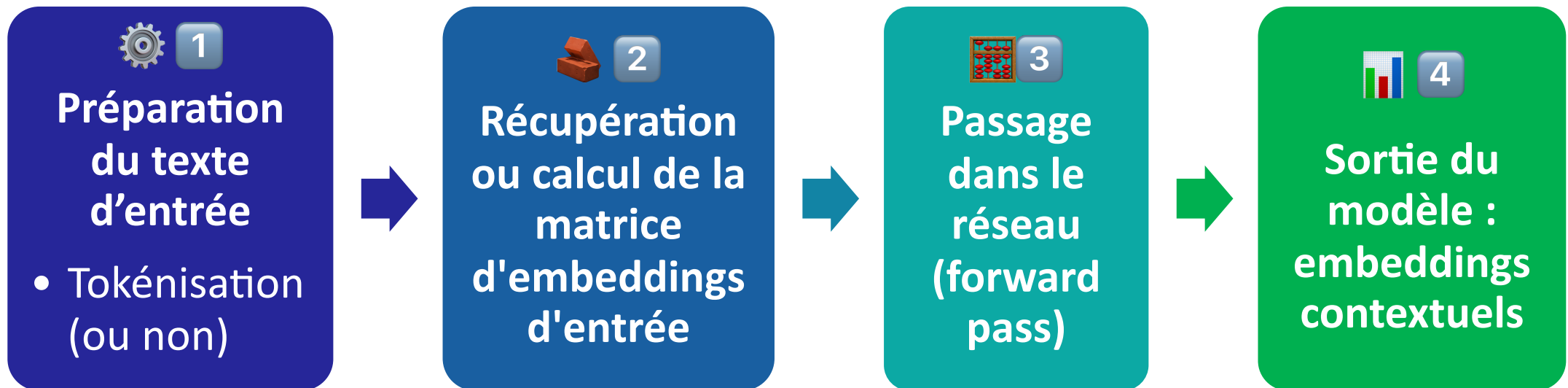
la matrice d'embedding $E \in \mathbb{R}^{|V| \times d}$

= un grand **dictionnaire de vecteurs de mots (ou tokens)** affiné automatiquement par l'apprentissage.

Etapes d'entrainement d'un modèle à embeddings



Etapes de l'inférence dans un modèle à embeddings



Deux types de modèles et leurs embeddings

Modèles dédiés aux embeddings 🎯

Vocation principale : apprendre une table de vecteurs

Word2Vec (2013) : prédire le mot ou le contexte (Skip-gram, CBOW)

GloVe (2014) : factorisation de matrices de cooccurrences

FastText (2016) : sous-mots, morphologie

🔑 **But** : produire des représentations vectorielles utilisables directement (similarité, analogies, clustering, etc.).

Modèles de langage générateurs d'embeddings 🌐

Vocation principale : modéliser le langage (prédiction, génération)

ELMo (2018) : LSTM contextuels, plusieurs vecteurs par mot

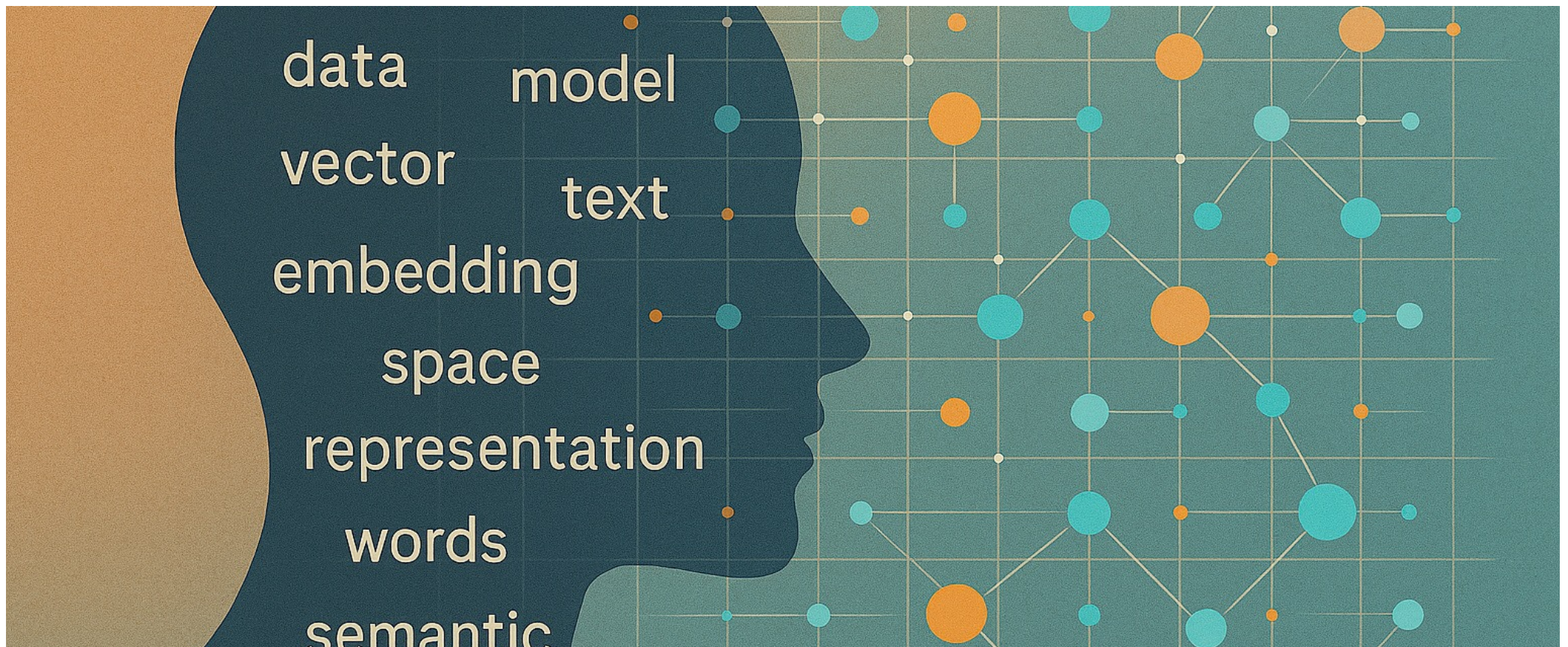
BERT (2018) : contextual embeddings issus des transformers

GPT (2018+) : génération, mais hidden states = embeddings

← **Sentence-BERT (2019)** : adaptation de BERT pour créer des embeddings de phrases

🔑 **But** : comprendre ou générer du texte →
embeddings = sous-produit qu'on réutilise pour la recherche, la similarité, etc.

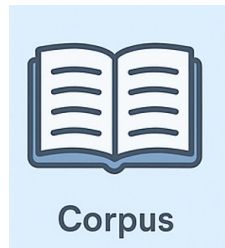
4 – Modèles dédiés aux word embeddings statiques



Word2Vec (Mikolov et al., 2013)



- Objectif :
 - Construire des vecteurs de mots à partir de corpus en fonction des **mots voisins**
- Paramètre:
 - **taille de la fenêtre contextuelle**
- Réseau de neurone simple à deux couches :
 - poids ajustés en minimisant les erreurs de prédiction



Efficient Estimation of Word Representations in Vector Space

Tomas Mikolov

Google Inc., Mountain View, CA
tmikolov@google.com

Kai Chen

Google Inc., Mountain View, CA
kaichen@google.com

Greg Corrado

Google Inc., Mountain View, CA
gcorrado@google.com

Jeffrey Dean

Google Inc., Mountain View, CA
jeff@google.com

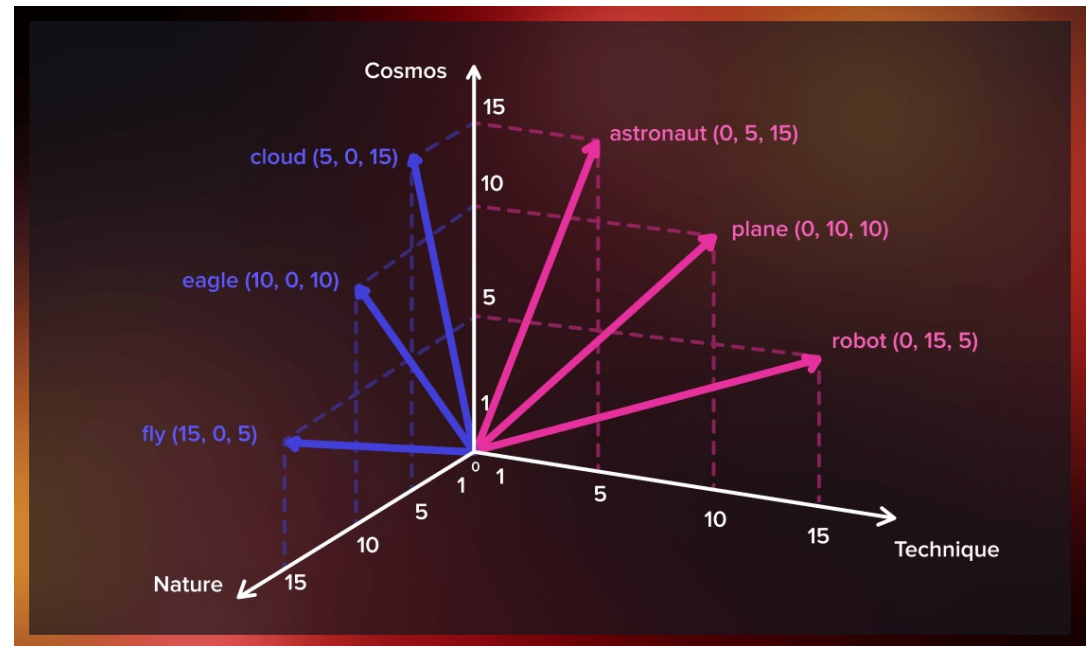
Abstract

We propose two novel model architectures for computing continuous vector representations of words from very large data sets. The quality of these representations is measured in a word similarity task, and the results are compared to the previously best performing techniques based on different types of neural networks. We observe large improvements in accuracy at much lower computational cost, i.e. it takes less than a day to learn high quality word vectors from a 1.6 billion words data set. Furthermore, we show that these vectors provide state-of-the-art performance on our test set for measuring syntactic and semantic word similarities.

Mikolov, T., Chen, K., Corrado, G., & Dean, J. (2013).
Efficient Estimation of Word Representations in Vector Space.
Proceedings of ICLR 2013. <https://arxiv.org/pdf/1301.3781>

Principe général

- Chaque mot → un seul vecteur fixe, appris à partir de très grands corpus
- Le vecteur ne dépend pas du contexte général, mais encode une sorte de “moyenne” de ses usages
- Le contexte est ici limité aux mots voisins explorés selon la fenêtre définie :
 - ex: 2 mots avant et après

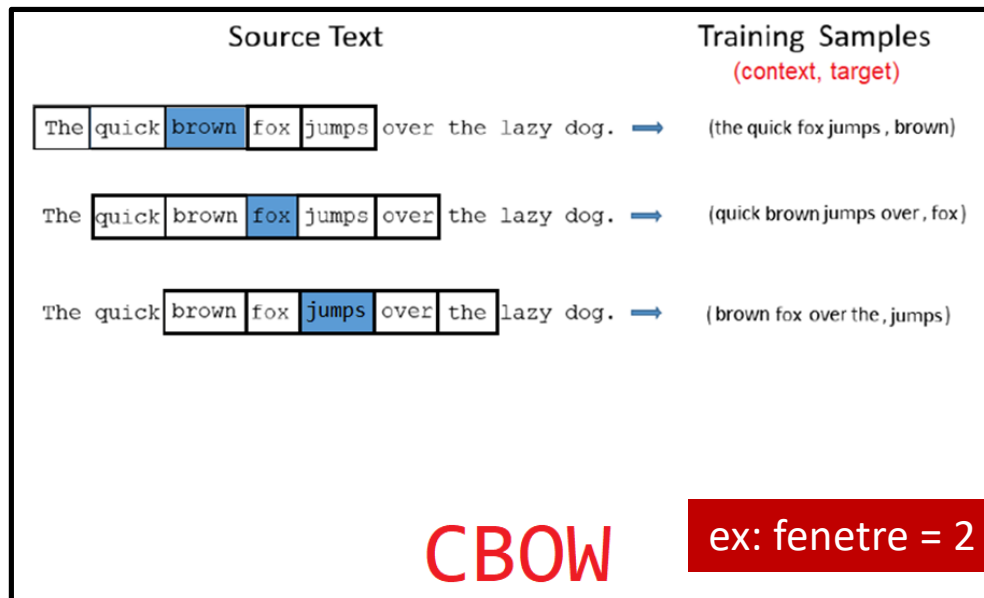


source : <https://serokell.io/blog/word2vec>

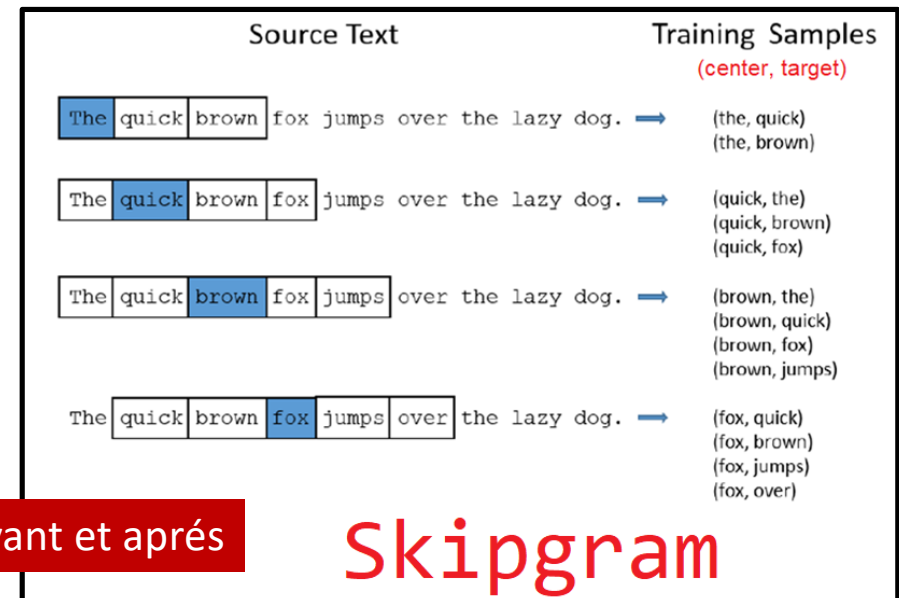
Même si l'on ne parle pas d'encodage d'embeddings contextuels, le contexte est bien utilisé pendant l'entraînement

Word2Vec: Deux techniques d'entrainement

- **CBOW (Continuous Bag of Words)**
: prédire un mot manquant à partir du contexte = "remplir le blanc"
 - ex: 1 mot à partir de 4 autres mots



- **Skip-gram** : prédire le contexte à partir du mot. = "deviner le contexte"
 - ex: probabilité de plusieurs mots environnants à partir d'un mot donné



Choix Cbow ou skip-Gram ?

CBOW

- Combine (moyenne) les vecteurs du contexte pour prédire 1 mot
 - Donc 1 prédiction par fenêtre
- Moins coûteux →
 - Plus rapide, car moins de calculs
 - efficace sur gros corpus
- Tendence à lisser les représentations (bon pour mots fréquents)

Skip-Gram

- Pour chaque mot cible, on doit prédire tous les mots du contexte
 - Donc plusieurs prédictions par fenêtre.
- ⇒ Plus coûteux →
 - Plus lent, surtout sur gros corpus
 - mieux sur petits corpus
- Apprend mieux les vecteurs des mots rares, car chaque paire (mot, contexte) est utilisée

Dans les deux cas, on obtient :

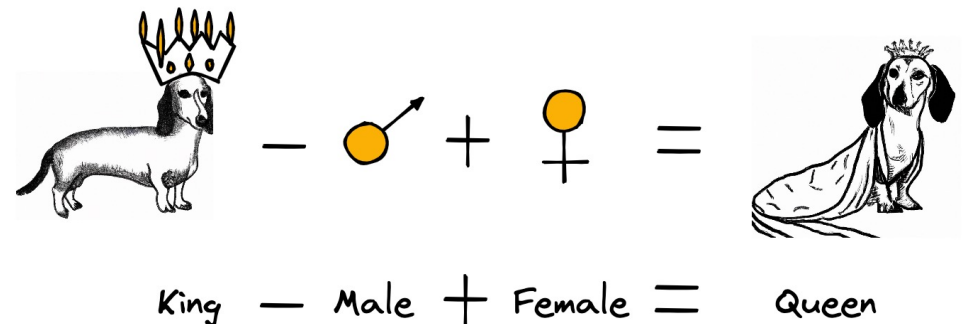
- Une matrice d'embedding (vecteurs denses pour chaque mot)
- Des similarités exploitables

🌟 Apports de Word2Vec

- Passage du discret au continu
 - Chaque mot = vecteur dense de dimension faible (100–300)
 - Représentation compacte et efficace
- Captation des similarités sémantiques
- Analogies et régularités vectorielles
- Efficacité d'entraînement

Impact majeur

- A ouvert la voie aux embeddings modernes et aux LLM
- Utilisé massivement dans NLP



Résultat de l'entraînement d'un modèle Word2Vec

- Le modèle apprend une grande matrice d'embeddings notée W_{in} de taille : $|V| \times d$

où :

- $|V|$ = taille du vocabulaire (min 30000 mots)
- d = dimension des embeddings (100 à 1024)

- Chaque ligne de cette matrice correspond à un mot du vocabulaire.
👉 Cette ligne est le vecteur du mot appris

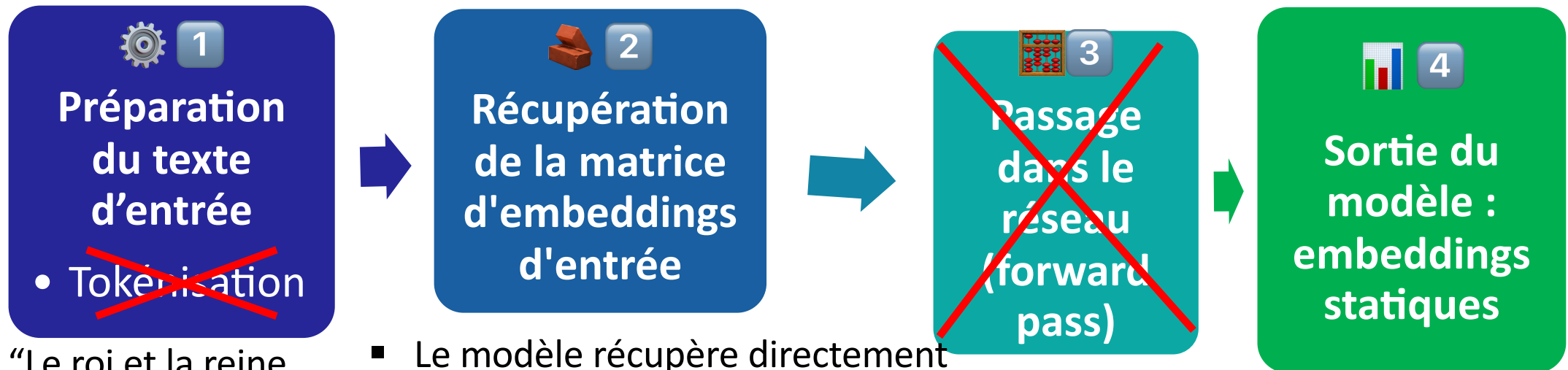
$$W_{in} = \begin{bmatrix} \text{roi} & \rightarrow & [0.21, 0.45, 0.10] \\ \text{reine} & \rightarrow & [0.25, 0.48, 0.09] \\ \text{homme} & \rightarrow & [0.33, 0.12, 0.56] \\ \text{femme} & \rightarrow & [0.34, 0.15, 0.51] \\ \text{château} & \rightarrow & [0.80, 0.10, 0.20] \end{bmatrix}$$

**La matrice W_{in} est figée après
l'entraînement**

Chaque mot est représenté par son
index = sa ligne dans la matrice

Spécificités de l'inférence dans Word2Vec

Le contexte n'est pas utilisé
il n'est utilisé que lors de l'entraînement



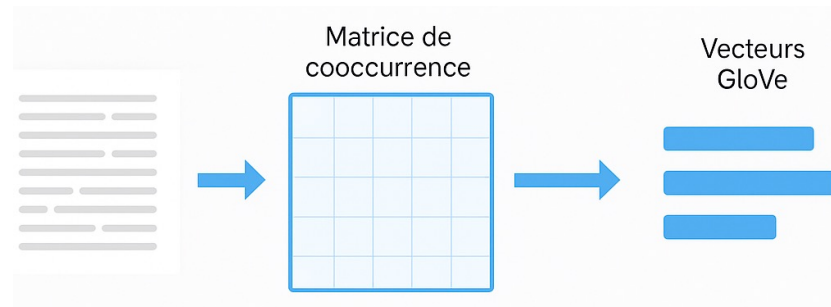
"Le roi et la reine
vivent au chateau" →
[roi, reine, château]
Les mots absents du
vocabulaire sont
ignorés

- Le modèle récupère directement les embeddings dans la matrice W_{in} :
 $v(\text{roi}) = [0.21, 0.45, 0.10]$
 $v(\text{reine}) = [0.25, 0.48, 0.09]$
 $v(\text{château}) = [0.80, 0.10, 0.20]$

Simple recherche dans une
table de vecteurs

Modèle GloVe (Pennington & al., 2014)

- Global Vectors for Word Representation
- Basé sur une matrice de cooccurrence (globale) :
 - combien de fois un mot i apparaît avec j ?
- Chaque vecteur encode les statistiques globales de coapparition des mots
- Combine approche locale (Word2Vec) + globale (cooccurrence)



Jeffrey Pennington, Richard Socher, and Christopher Manning. 2014. *GloVe: Global Vectors for Word Representation*. In *Proceedings of the 2014 Conference on Empirical Methods in Natural Language Processing (EMNLP)*, pages 1532–1543, Doha, Qatar. Association for Computational Linguistics.
<https://aclanthology.org/D14-1162.pdf>

Modèle FastText (Bojanowski & al. 2017)

Principe : enrichir les embeddings par les n-grams de caractères

- Un n-gram est une séquence continue de n-éléments (mots, caractères, ...) : fenêtre "glissante" de longueur n sur une phrase
- Exemple : Le chat dort sur le canapé

Type de n-gram	Exemple	Contient...
1-gram (unigramme)	[Le], [chat], [dort], [sur], [le], [canapé]	1 mot
2-gram (bigramme)	[Le chat], [chat dort], [dort sur], [sur le], [le canapé]	2 mots consécutifs
3-gram (trigramme)	[Le chat dort], [chat dort sur], [dort sur le], [sur le canapé]	3 mots consécutifs
4-gram (quadrigramme)	[Le chat dort sur], [chat dort sur le], [dort sur le canapé]	4 mots consécutifs

Modèle FastText (Bojanowski & al. 2017) *fast*Text

Vecteur d'un mot = somme des vecteurs de ses **n-grams de caractères**

Exemple : « jouer » → <jo, jou, oue, uer, er>

Chaque n-gram a un vecteur

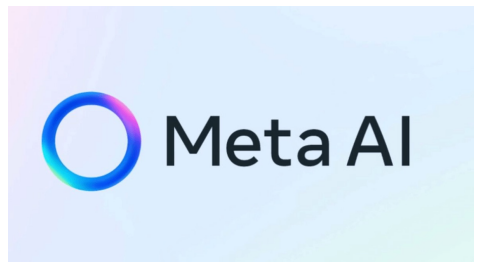
$v(\text{jouer}) = \text{somme}(v(<\text{jo}>), v(\text{jou}), v(\text{oue}), v(\text{uer}), v(\text{er}))$

✓ Avantages :

- Gère les mots rares ou inconnus (**OOV = Out-Of-Vocabulary**)
- Prend en compte la morphologie (préfixes, suffixes)
- Compatible avec Word2Vec (CBOW, Skip-gram)



facebook AI Research



Bojanowski, P. et al. (2017) 'Enriching Word Vectors with Subword Information', *Transactions of the Association for Computational Linguistics*, 5, pp. 135–146. Available at: https://doi.org/10.1162/tacl_a_00051.

Mise en pratique
Word2Vec/fastText
avec Gensim



Gensim: bibliothèque python pour les embeddings

- **Entraîner** ses propres modèles (*Word2Vec*, *FastText*)
- **Charger** des modèles pré-entraînés (*KeyedVectors*)
- **Explorer** les relations entre mots
Similarité cosinus
 - Recherche de voisins (`most_similar`)
 - Analogies vectorielles ($\text{roi} - \text{homme} + \text{femme} \approx \text{reine}$)

- Un seul outil pour plusieurs modèles

- Word2Vect
- FastText



```
#Installation  
pip install gensim
```



Word2Vec/fasttext avec Gensim : étapes

1. Préparer le corpus

Texte brut → segmentation en phrases → tokenisation en mots

```
corpus= [ ["le", "chat", "mange", "des", "croquettes"], ... ]
```

2. Créer et entraîner le modèle

```
model = Word2Vec(sentences=..., vector_size=..., ..., ...)  
#ou model = FastText(sentences
```

3. Explorer les vecteurs

```
#Récupérer un vecteur : model.wv["chat"]  
#Comparer des mots : model.wv.most_similar("chat")
```

Si on utilise un modèle préentraîné : `model = KeyedVectors.load("fichier.kv")`

Attention: on a directement `model` (pas besoin de `model.wv`)

Paramètres d'entrainement Word2Vec/FastText

- **sentences**: liste de tokens List[List[str]]
- **vector_size** : dimension des vecteurs (50, 100, 300...)
- **window** : taille de la fenêtre de contexte
- **min_count** : filtre les mots rares (ignore les mots qui apparaissent <min_count)
- **sg** : choix de l'algorithme
 - 0 → CBOW (*plus rapide, corpus grand*)
 - 1 → Skip-Gram (*meilleur sur petits corpus*)
- **epochs** : nombre d'itérations d'apprentissage (plus pour les petits corpus!)
- **workers** : nombre de threads CPU pour accélérer (optionnel)

```
from gensim.models import Word2Vec

model = Word2Vec(
    sentences=corpus, # corpus
    vector_size=100, # taille vecteur
    window=5, # fenêtre contexte
    min_count=1, # filtre
    sg=1, # 0=CBOW, 1 =Skip-Gram
    epochs=5, # itérations
)
```

Utilisations du modèle

- Obtenir un vecteur dense

```
#model.wv[word: str] -> numpy.ndarray
```

```
model.wv["chat"]
```

```
# ex: array([ 0.12, -0.03, 0.45, ..., 0.07], dtype=float32)
```

- Mesurer la similarité cosinus entre deux mots

entre 0 et 1

```
# model.wv.similarity(word1: str, word2: str) -> float
```

```
print(model.wv.similarity("chat", "chien"))
```

```
# ex 0.87
```

- Calculer la distance cosinus entre plusieurs mots

distance cosinus =
1 - similarité_cosinus

```
#model.wv.distances(word: str, other_words: list[str]) -> numpy.ndarray
```

```
print(model.wv.distances("chat", ["chien", "oiseau"]))
```

```
# ex: array([0.13, 0.42])
```

Plus proches voisins

```
# model.wv.most_similar(  
    list[str], topn: int = 10) -> list[tuple[str, float]]
```

- Trouver les 3 plus proches voisins d'un mot

```
model.wv.most_similar("chat", topn=3)  
#ex : [("chien", 0.87),  
       ("souris", 0.81),  
       ("animal", 0.78)]
```



Similarité cosinus

Analogies (type roi-homme+femme?)

```
# model.wv.most_similar(  
    positive:str | list[str] | dict[str, float],  
    negative:str | list[str] | dict[str, float] = None,  
    topn: int = 10  
) -> list[tuple[str, float]]
```

```
model.wv.most_similar(positive=["roi", "femme"],  
                      negative=["homme"], topn=3)
```

```
# ex : [("reine", 0.89),  
        ("princesse", 0.78),  
        ("souveraine", 0.75)]
```

Cherche les mots dont le vecteur est le plus proche de
cette combinaison linéaire
 $\text{vect}(\text{"roi"}) - \text{vect}(\text{"homme"}) + \text{vect}(\text{"femme"})$

Interprétation d'une analogie

- $\text{vect}(\text{"roi"}) - \text{vect}(\text{"homme"}) + \text{vect}(\text{"femme"})$
- On fait une opération géométrique sur les mots :
 - On retire le sens “homme” du mot “roi”
 - On obtient un vecteur auquel on ajoute le sens “femme”
 - On cherche alors les mots similaires à ce vecteur calculé
 - Le mot qui se trouve le plus proche de ce nouveau point dans l'espace est par exemple “reine”.

Le modèle a appris que roi/homme et reine/femme sont des relations analogues

Exemple

corpus jouet

```
from gensim.models import Word2Vec
sentences = [
    ["le", "chat", "mange", "des", "croquettes"],
    ["le", "chien", "dévore", "des", "os"],
    ["un", "chat", "est", "un", "animal", "domestique"],
    ["un", "chien", "est", "un", "animal",
     "domestique"]]
# Entraînement du modèle
model = Word2Vec(
    sentences=sentences,
    vector_size=5,      # taille du vecteur dense
    window=2,           # contexte de 2 mots
    min_count=1,        # garder tous les mots
    sg=1,               # Skip-Gram
    epochs=200          # itérations
)
# Affichage des vecteurs appris
for word in model.wv.index_to_key:
    print(f"{word:12s} -> {model.wv[word]}")
```

```
un          -> [-0.01702737  0.01183738  0.11816302  0.18594542 -0.19333751]
domestique  -> [-0.14548221  0.13289596  0.18694769 -0.09737356 -0.08016656]
animal      -> [ 0.14577308 -0.02811422 -0.08483662  0.1350295  -0.10190821]
est         -> [-0.03806889  0.0602668  0.02701619 -0.16348517 -0.19588302]
chien       -> [0.14524257 0.10613511 0.14463858 0.02077042 0.12558773]
des         -> [-0.0728342  -0.01310823  0.12506396 -0.14769511 -0.07970104]
chat        -> [-0.15401435 -0.01574212  0.19910468 -0.14510307 -0.05102513]
le          -> [-0.0409691  0.16716252 -0.10931668  0.0067923  -0.09430145]
os          -> [-0.19535488  0.1054064  -0.16874677 -0.0847601  -0.0021147 ]
dévore      -> [-0.00795532 -0.14904255  0.19920957  0.10328539  0.1844665 ]
croquettes  -> [-0.16471216  0.09283617 -0.07887904  0.0185333  0.16983849]
mange       -> [-0.09133989  0.09258478 -0.13255827 -0.0693955  0.18734096]
```

A vous de jouer : TP1-WordEmb.pynb



- Récupérez le notebook TP1-WordEmb.pynb
- Récupérez le modèle pré-entraîné réduit :

Deux fichiers à télécharger et à placer dans le même répertoire que le notebook:

- cc.fr.100k.kv (structure du modèle : le vocabulaire (les mots connus), les paramètres (dimension, etc.), et les métadonnées.)
- cc.fr.100k.kv.vectors.npy (matrice numérique des vecteurs)

Partie A — Entraînement de modèles Word2Vec et FastText from scratch (corpus jouet)

- Visualisation des vecteurs
- Exploration des similarités
- Variation des paramètres
- Comparaison Word2Vec-FastText



Partie B — Modèle pré-entraîné FastText (version réduite)

- Analogies
- Problème de polysémie

Certaines cellules de code sont juste à tester, parfois en faisant varier certains paramètres, d'autres sont à compléter.
Répondez aussi à plusieurs questions 🤔 dans les cellules de texte.

Limites des embeddings statiques

- Vecteurs appris une fois pour toute
 - Entrainement sur un corpus fixe
 - Pas d'adaptation, pas d'évolution, pas d'incrémentalité
- Polysémie non gérée : un seul vecteur pour plusieurs sens



5 – Modèles d'embeddings contextuels



Principe des embeddings contextuels

Ce qui change à partir de 2018

- Le vecteur d'un mot dépend de son contexte :
 - plusieurs vecteurs pour chaque mot
- Construction basée sur des réseaux de neurones profonds séquentiels
 - LSTM (**Long Short Term Memory**) puis Transformers et le mécanisme d'attention
- Vecteurs évolutifs : Ne sont pas fixés une fois pour toutes
- Sont calculés dynamiquement à partir d'une phrase entière

Captation des nuances de sens et de syntaxe

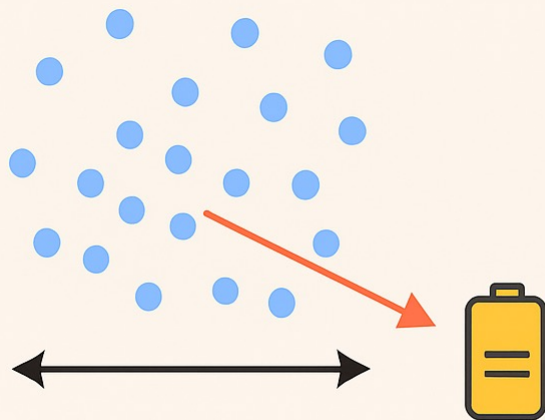
Notion d'embedding dynamique

- Plusieurs vecteurs par mot :
Chaque occurrence d'un mot
a son propre vecteur

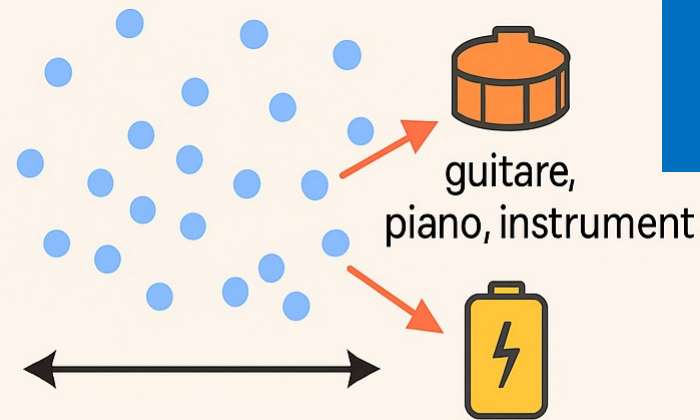
Il joue de la *batterie*.

La batterie de mon
téléphone est faible.

Embeddings statiques (Word2Vec, GloVe)



Embeddings dynamiques (BERT)



On parle de
contextual
embeddings

Au-delà des embeddings : la vocation des modèles de langage

■ Comprendre le langage

Analyse du contexte

Désambiguïsation du sens

■ Prédire et générer

Complétion de texte

Traduction, résumé, dialogue

■ Produire des représentations utiles

Les *embeddings* sont un **sous-produit**

Réutilisés pour la recherche sémantique, la classification, etc.



Word2Vec, GloVe = modèles d'embeddings statiques

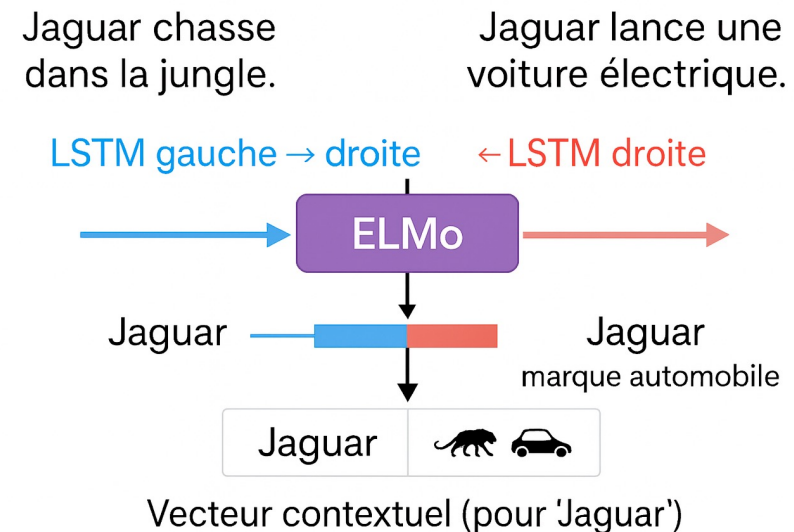


ELMo, BERT = modèles de langage qui génèrent aussi des embeddings

ELMo (Matthew & al., 2018)

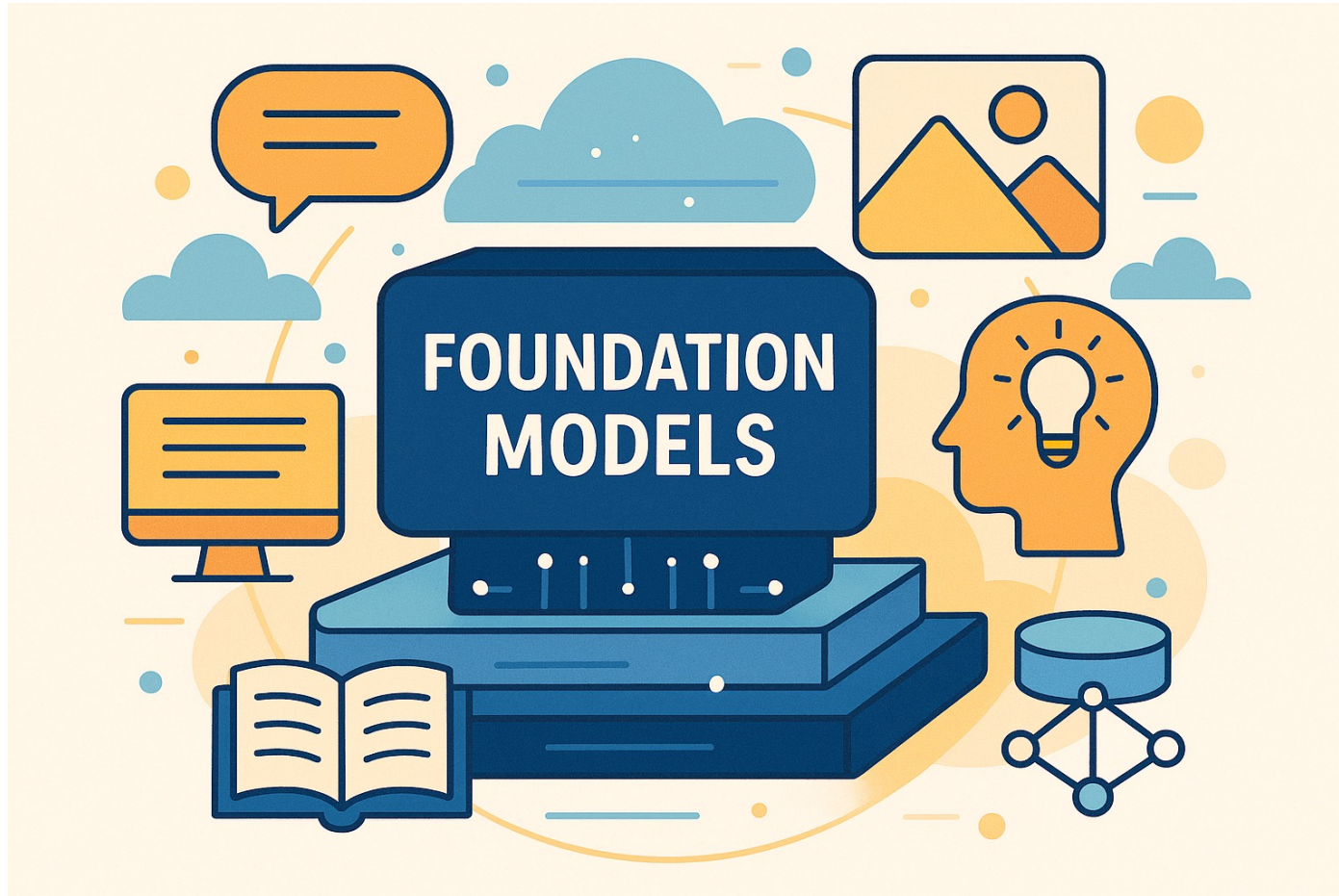
Embeddings from Language Models

- Basé sur des réseaux de neurones LSTM bidirectionnels :
 - LSTM (Long Short-Term Memory) = réseau séquentiel avec une mémoire qui permet de retenir les informations importantes et d'oublier le reste.
 - Deux parcours : gauche → droite et droite → gauche.
- Combine les représentations des deux directions pour obtenir un vecteur riche en contexte
- Produit un vecteur qui dépend de la phrase entière



Peters, Matthew E., et al. (2018). Deep contextualized word representations. In: *Proceedings of the 2018 Conference of the North American Human Language Technologies (NAACL-HLT 2018)*, Volume 1 (Long Papers), pp. 2227–2237.
<https://aclanthology.org/N18-1202>

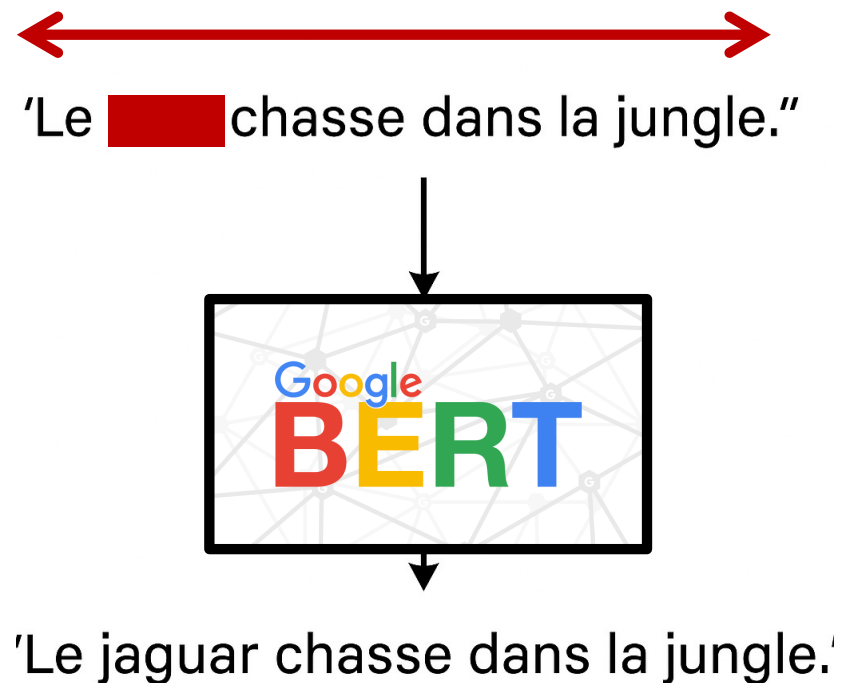
BERT : le choc des Transformers



BERT

Bidirectional Encoder Representations from Transformers (Devlin & al. 2019)

- Remplace les LSTM par les **Transformers** (self-attention)
- Apprentissage auto-supervisé : **Masked Language Modeling** (**Objectif**= prédire des mots masqués)
- Produit des embeddings puissants et contextuels



Devlin, J., Chang, M.-W., Lee, K., & Toutanova, K. (2019).
BERT: Pre-training of Deep Bidirectional Transformers for Language Understanding.
Proceedings of NAACL-HLT 2019, pp. 4171-4186.
arXiv:1810.04805

BERT: variantes et améliorations

Plusieurs modèles améliorent les performances de BERT initial :

- RoBERTa : plus de données, meilleur entraînement.
- DistilBERT : version légère et rapide.
- ALBERT : paramètres partagés → efficacité.
- **CamemBERT** : adaptation française de BERT, entraînée sur 138 Go de texte OSCAR (Inria + Facebook AI).
- Tous gardent le principe d'embeddings contextuels.

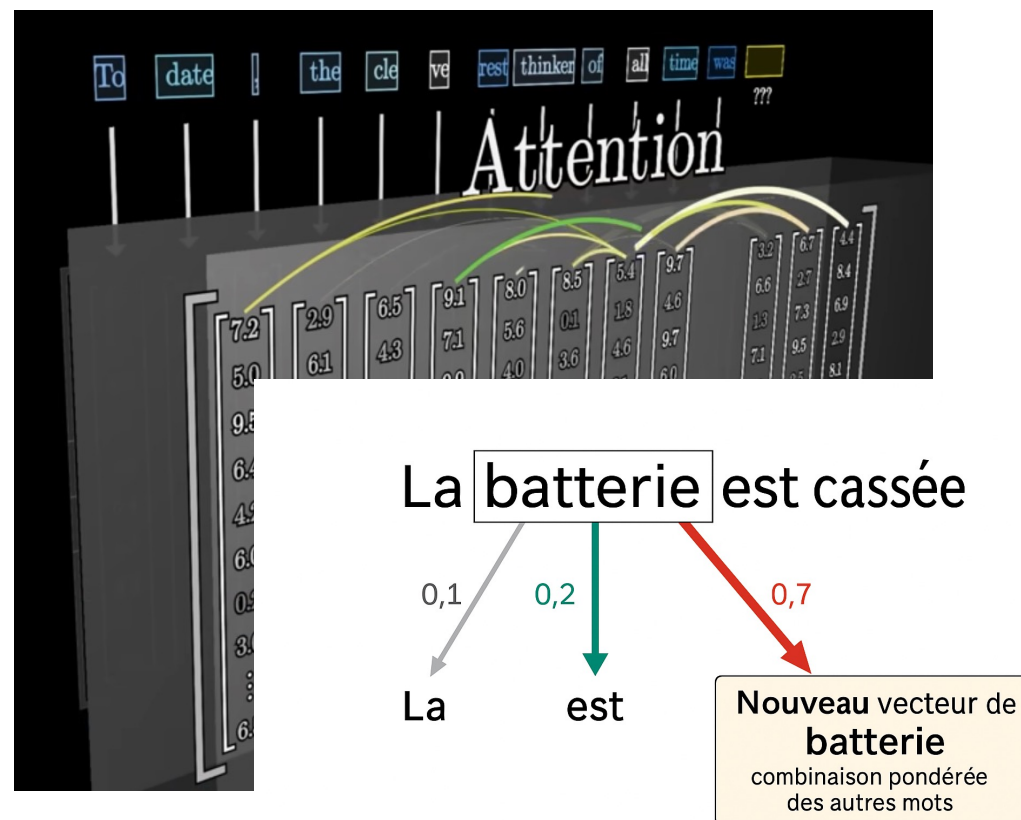
"Attention is all we need" (Vaswani & al., 2017)

- Mécanisme central :

l'attention

→ chaque mot regarde tous les autres et pondère leur importance pour construire son embedding

→ calcul de **poids d'attention** (*attention scores*)



Vaswani, A. et al. (2023) 'Attention Is All You Need'. *31st Conference on Neural Information Processing Systems (NIPS)*. Advances in Neural Information Processing Systems. Vol. 30. Curran Associates, Inc. [arXiv:1706.03762](https://arxiv.org/abs/1706.03762).

Une video explicative : https://www.youtube.com/watch?v=eMlx5fFNoYc&list=PLZHQObOWTQDNU6R1_67000Dx_ZCJB-3pi&index=7

Deux types d'Embeddings dans les transformers encodeurs (ex: BERT)



Embeddings de tokens

Les embeddings ne portent plus sur des mots (Word2Vec) mais sur des tokens =

unités lexicales élémentaires

(caractères, sous-mots ou parfois mots entiers)

- Résolution du problème de mots inconnus (OOV)
- Meilleure généralisation morphologique (prefixes, ...)



Embeddings de position

Définissent la position des tokens dans la phrase

ce vecteur déplace légèrement le token selon où il se trouve dans la phrase, pour que deux occurrences du même mot aient des vecteurs proches mais distincts



sans ces embeddings le modèle produit le même embedding pour

"le chat mange la souris"

et *"la souris mange le chat"*

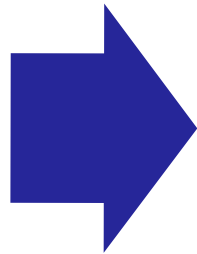
Spécificités de l'inférence dans un Transformer encodeur



1

Préparation du texte d'entrée

- Tokénisation



2

Calcul de la matrice d'embeddings d'entrée

“Le roi et la reine vivent au chateau” →

La phrase est convertie en séquence de tokens avec ajout de marqueurs spéciaux

[CLS], Le, roi, et, la, reine, vivent, au, château, [SEP]

token de classification

token de séparation

Chaque token est remplacé par son identifiant numérique dans le vocabulaire **SentencePiece (pour Camembert)** (32 000 à 50 000 tokens selon le modèle)

vecteur `input_ids` de longueur 10

Token	ID
[CLS]	0
Le	1278
roi	5304
...	...
château	8125
[SEP]	2

Spécificités de l'inférence dans un Transformer encodeur



2

Calcul de la matrice d'embeddings d'entrée

1. Création de la matrice d'embeddings de tokens **E_token**

Taille = [Vocab_size × Hidden_dim] → par ex. [32 000 × 768]

récupérés dans la Table de lookup des embeddings à partir des ID de tokens

2. Création de la matrice d'embeddings de position **E_pos**

Taille = [max_seq_len × Hidden_dim] → [514 × 768] pour

CamemBERT

Chaque ligne correspond à une position dans la séquence (ligne 0=position du premier token, ...)

Ces vecteurs indiquent la position du token dans la phrase (appris)

3. Les deux matrices sont sommées

$X = E_{\text{token}} + E_{\text{pos}}$

les vecteurs de position
n'ont pas de signification
explicite

Token	Embedding (E_token)
Le	[0.10, 0.25, -0.30, 0.05]
roi	[0.20, 0.40, -0.10, 0.15]
reine	[0.18, 0.42, -0.12, 0.13]
château	[0.50, -0.10, 0.25, 0.05]

Position	E_pos
0	[0.01, 0.02, 0.03, 0.04]
1	[0.05, 0.10, 0.15, 0.20]
2	[0.09, 0.18, 0.27, 0.36]
3	[0.13, 0.26, 0.39, 0.52]

Exemples de matrices d'embeddings de tokens dans un Transformer encodeur

[d1] Le chat mange des croquettes

Taille séquence = 7 ; dimension = 768

00 <s>	→	[0.0087 0.0363 0.0243 -0.0603 0.0025 -0.004 -0.0248 0.0653]
01 _Le	→	[-0.0803 -0.5432 0.0693 -0.0847 -0.0808 -0.4355 0.153 0.028]
02 _chat	→	[0.1432 -0.3108 0.3444 -0.0643 0.0886 0.2922 -0.1484 0.0214]
03 _mange	→	[-0.0287 -0.3534 0.0603 -0.1681 -0.1689 -0.5769 -0.054 0.0368]
04 _des	→	[0.0219 0.0805 0.0063 -0.2048 -0.0684 -0.2624 -0.069 0.044]
05 _croquettes	→	[0.0378 0.0953 0.3981 -0.0812 0.039 0.073 -0.0946 0.0848]
06 </s>	→	[-0.06 0.1087 0.048 -0.2047 -0.1651 -0.0672 -0.0953 0.0367]

[d3] Un chat est un animal domestique

Taille séquence = 8 ; dimension = 768

00 <s>	→	[0.0165 0.0239 0.0124 -0.0503 0.0074 0.0018 -0.0176 0.0584]
01 _Un	→	[0.055 -0.3241 -0.2755 -0.1839 0.1302 -0.5228 -0.1555 -0.095]
02 _chat	→	[0.1577 -0.0803 0.3833 -0.1008 0.1115 0.2819 -0.1008 0.0919]
03 _est	→	[0.1064 -0.3084 -0.3182 -0.1878 0.0274 -0.6565 0.0491 -0.0629]
04 _un	→	[0.1376 -0.225 -0.3191 0.0019 -0.0301 -0.139 -0.0411 0.0201]
05 _animal	→	[0.0678 -0.2288 -0.1005 -0.0376 0.1428 -0.0086 -0.2718 -0.0126]
06 _domestique	→	[0.0657 -0.2859 0.6732 -0.0518 -0.0976 0.417 -0.3391 -0.0148]
07 </s>	→	[-0.0289 0.1522 -0.0005 -0.2202 -0.1277 0.0104 -0.0769 0.0627]

Spécificités de l'inférence dans un Transformer encodeur



2

Calcul de la matrice d'embeddings d'entrée

Token	Position	Token embedding	Position embedding	Somme (entrée du modèle)
Le	0	[0.10, 0.25, -0.30, 0.05]	[0.01, 0.02, 0.03, 0.04]	[0.11, 0.27, -0.27, 0.09]
roi	1	[0.20, 0.40, -0.10, 0.15]	[0.05, 0.10, 0.15, 0.20]	[0.25, 0.50, 0.05, 0.35]
reine	2	[0.18, 0.42, -0.12, 0.13]	[0.09, 0.18, 0.27, 0.36]	[0.27, 0.60, 0.15, 0.49]
château	3	[0.50, -0.10, 0.25, 0.05]	[0.13, 0.26, 0.39, 0.52]	[0.63, 0.16, 0.64, 0.57]

La matrice d'entrée est définie par la somme des deux matrices d'embeddings de token et de position

$$X = E_{\text{token}} + E_{\text{pos}}$$

Spécificités de l'inférence dans un Transformer encodeur



3

Passage dans le réseau (forward pass)

La matrice d'entrée X passe à travers :

- 12 blocs d'auto-attention
- suivis de couches feed-forward (Perceptron Multi Couches)
- avec normalisation et résidus à chaque étape

Ce processus raffine les vecteurs, qui deviennent contextualisés :

chaque token est ainsi représenté *en tenant compte des autres*



4

Sortie du modèle

Sortie de la dernière couche →

last_hidden_state [seq_len x 768]

- Chaque ligne correspond à un token de la phrase
- Chaque vecteur (768 dimensions) est contextualisé

Classification (phrase entière)

→ on prend le vecteur du token [CLS] :
vecteur contextuel de la phrase

• Pour une recherche sémantique

→ on peut moyenner tous les vecteurs (mean pooling)

BERT : les matrices d'embeddings de tokens

Matrices des embeddings de tokens (token x dimensions)

- Une matrice par entrée (phrase)
 - T lignes: tokens
 - H colonnes : dimensions cachées (768 pour CamemBERT-base)
- La dernière version (dernière couche) : **last_hidden state**

Tokens ajoutés

<s> = *start of sentence* → [CLS] classification token
</s> = *end of sentence* → [DEP]

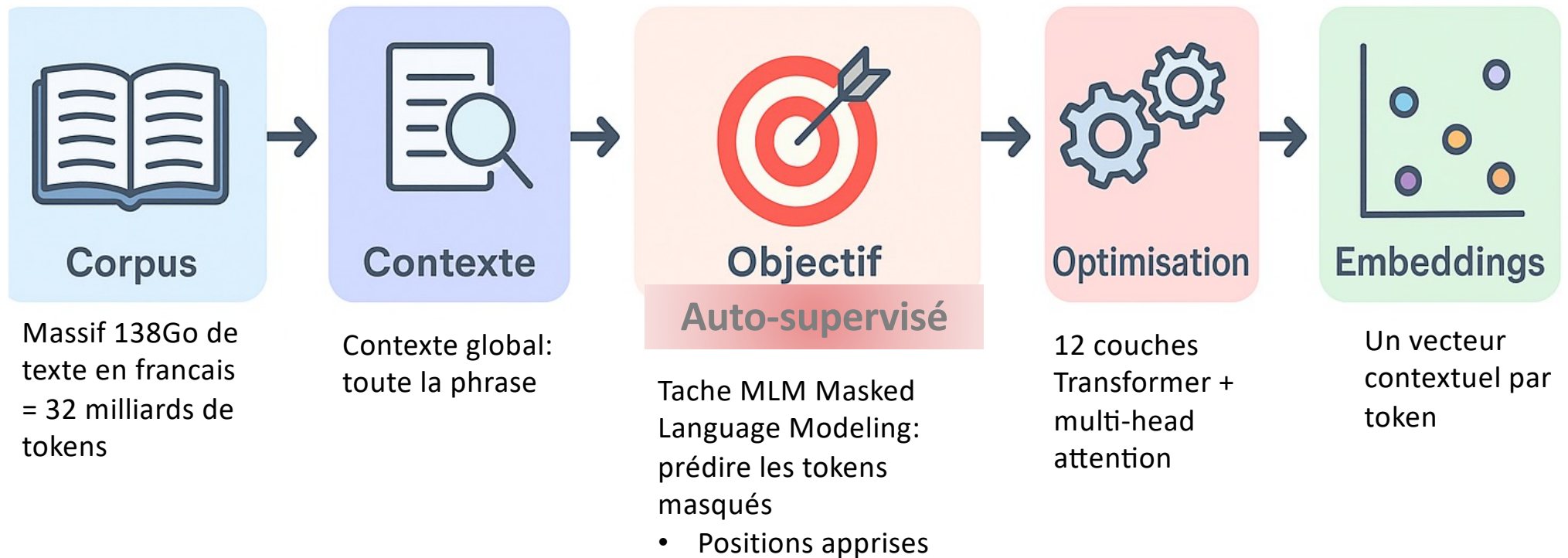
Hidden-
states

La batterie est cassée dans la salle de répétition.
Taille séquence : 12 Dimension vecteurs : 768

	token	dim1	dim2	dim3	dim4	dim5	dim6	dim7	dim8	dim9	dim10
0	<s>	0.03	0.12	0.10	-0.09	0.03	-0.02	0.06	0.19	0.13	0.07
1	__La	-0.07	0.30	0.10	-0.03	0.15	-0.24	0.14	0.01	0.14	0.08
2	__batterie	0.04	-0.07	-0.05	0.03	0.07	-0.14	0.11	-0.05	0.09	0.13
3	__est	-0.03	0.02	-0.31	-0.11	-0.06	-0.16	0.18	-0.10	0.11	0.23
4	__cassée	0.05	0.15	0.15	0.08	-0.12	-0.05	0.03	0.09	0.03	0.17
5	__dans	0.08	0.10	-0.09	0.11	0.00	-0.09	0.03	0.06	-0.01	0.20
6	__la	-0.06	0.32	0.08	0.11	0.10	-0.09	0.12	0.03	0.06	0.10
7	__salle	0.01	0.00	-0.22	0.05	0.13	0.01	0.06	-0.11	0.06	0.07
8	__de	-0.10	0.35	-0.10	-0.01	0.05	-0.03	0.10	-0.03	0.15	0.17
9	__répétition	-0.13	-0.13	0.17	0.02	0.11	-0.07	-0.13	0.09	0.14	0.07
10	.	-0.05	0.11	0.12	-0.14	-0.02	-0.00	0.01	0.13	0.09	0.04
11	</s>	-0.08	0.20	0.10	-0.12	-0.03	-0.06	0.08	0.12	0.12	0.07

avec CamemBERT-base

Spécificités de l'entrainement d'un transformer encodeur (ex: CamemBERT)



📌 Les embeddings de position sont appris en même temps que les embeddings de token : ils permettent au modèle d'apprendre l'ordre des mots et la structure syntaxique en guidant l'attention

Embedding de phrase (sentence embedding)

- Créés uniquement si nécessaire - 3 méthodes possibles :

Méthode	Principe	Avantages 	Limites 	Cas d'usage
CLS	On prend le vecteur du token spécial <s> (équivalent [CLS]) à la sortie de la dernière couche	Simple, natif dans BERT, vecteur unique par phrase	Pas optimisé pour la similarité sémantique	Classification, NSP (Next Sentence Prediction)
Mean pooling	Moyenne des vecteurs de tous les tokens de la phrase (hors padding)	Plus robuste, capture mieux le « centre » du sens global	Peut diluer les informations fines	Similarité entre phrases, clustering
Pooler output	Vecteur CLS projeté par une couche linéaire + tanh (prévu pour NSP)	Vecteur normalisé, utilisé dans l'entraînement original	Entraîné pour NSP, peu pertinent pour la recherche sémantique	Tâches supervisées de classification

Embedding de phrase (sentence embedding)

■ Exemple

- vecteur pour "La batterie est cassée dans la salle de répétition."
- Taille séquence (nombre de tokens) =12
- Dimension des vecteurs = 768

	0	1	2	3	4	5	6	7	8	9
dim	dim1	dim2	dim3	dim4	dim5	dim6	dim7	dim8	dim9	dim10
CLS	0.029	0.123	0.1	-0.094	0.031	-0.016	0.062	0.19	0.128	0.073
Mean	-0.025	0.124	0.004	-0.009	0.035	-0.077	0.066	0.036	0.091	0.118
Pooler	-0.094	0.112	0.023	0.036	-0.063	0.161	0.043	0.109	-0.065	-0.025

Aucune solution n'est optimisée pour mesurer directement la similarité sémantique entre phrases.

Sentence-BERT (SBERT)

Modèle spécialisé pour embeddings de phrase

- C'est une adaptation de BERT (fine-tuning avec un objectif de similarité de phrases).
- Conçu spécifiquement pour **produire des sentence embeddings** de qualité.
- Résultat : un seul vecteur par phrase/document, optimisé pour la recherche sémantique, le clustering, etc.

Mise en pratique CamemBERT/SentenceBERT



La librairie python 🤗 Transformers

- Librairie Python incontournable pour manipuler des modèles pré-entraînés en NLP.
- Fournit :
 - **Tokenizers** (préparation des textes → tokens).
 - **Models** (chargement des poids et architecture).
 - **Pipelines** (API haut-niveau pour tâches directes : classification, NER, QA...).
- Compatible avec PyTorch & TensorFlow.
- Des milliers de modèles disponibles sur huggingface.co/models

CamemBERT

Importer le modèle et tokeniser

Importer le modèle et le tokenizer

```
from transformers import CamembertTokenizer, CamembertModel
tokenizer = CamembertTokenizer.from_pretrained("camembert-base")
model = CamembertModel.from_pretrained("camembert-base")
```

Encoder (tokenisation) une phrase

```
sentence = "La batterie est cassée."
```

```
inputs = tokenizer(sentence, return_tensors="pt")
```

objet de type BatchEncoding

```
# dictionnaire PyTorch
```

```
{'input_ids': tensor([[ 5, 61, 2629, 30, 17082, 9, 6]]),
 'attention_mask': tensor([[1, 1, 1, 1, 1, 1, 0]])}
```

id des tokens

tableau 2D [batch, vecteur [seq_len]

tableau 2D [batch, vecteur [seq_len]] avec :

1 pour les tokens réels

0 pour les tokens padding (token ajoutés pour uniformiser la taille)

CamemBERT

Passage dans le modèle

utilisation du modèle en inférence

```
camem.eval()
```

```
with torch.no_grad(): # pas de back propagation
```

#matrice des embeddings de position

taille =[max_len, 768]

```
pos_embed = camem.embeddings.position_embeddings.weight
```

#matrice des embeddings initiale des tokens

```
first_embed = camem.get_input_embeddings() .weight
```

taille =[vocab_size, 768]

CamemBERT

Passage dans le modèle

Passage dans le modèle (inférence)

```
outputs = model(**inputs, return_dict=True)
```

Le résultat est un dictionnaire

Objet spécial HuggingFace :
BaseModelOutputWithPoolingAndCrossAttentions

last_hidden_state : Matrice des embeddings de tokens.

Forme : [batch_size, seq_len, hidden_dim]

Exemple : [1, 8, 768] → 1 phrase, 8 tokens, vecteurs de 768 dimensions.



Principaux attributs

pooler_output (optionnel selon le modèle) : Embedding du token **[CLS]**, passé dans une petite couche supplémentaire (dense + tanh).

Forme : [batch_size, hidden_dim] → ex. [1, 768].

hidden_states (si *output_hidden_states=True*) : Liste des matrices pour **chaque couche** du modèle (y compris l'embedding initial).

Longueur : nb_layers + 1

Chaque élément : [batch_size, seq_len, hidden_dim].

attentions (si *output_attentions=True*)

Les poids d'attention pour chaque couche et chaque tête.

CamemBERT

Générer des embeddings de phrase

Récupérer les embeddings de tokens

```
last_hidden = outputs.last_hidden_state      # [batch, tokens, 768]
```

CLS : vecteur du token spécial <s>

```
cls_vec = last_hidden[:, 0, :]
```

Mean pooling : moyenne des vecteurs de tokens (hors padding)

Enlever la dimension batch (ex: [1, seq_len] -> [seq_len])

```
mask = inputs["attention_mask"].squeeze(0)
```

Ajouter une dimension colonne (ex: [seq_len] -> [seq_len, 1])

pour pouvoir multiplier chaque vecteur token (768 dim) par 1 ou 0

```
mean_vec = (hidden * mask.unsqueeze(-1)).sum(0) / mask.sum()
```

Pooler output : CLS projeté par une couche supplémentaire

```
pooler_vec = out.pooler_output.squeeze(0)
```

Le rôle de attention_mask

- Lors de la tokenisation, le tokenizer ajoute parfois du **padding** (tokens [PAD]) pour uniformiser les séquences d'un batch (même taille).
- `inputs["attention_mask"]` est un vecteur `[seq_len]` avec :
 - 1 pour les tokens réels
 - 0 pour les tokens padding
- Exemple pour une phrase de 5 tokens, padded à 8 :

```
attention_mask = [1, 1, 1, 1, 1, 0, 0, 0]
mask = inputs["attention_mask"].squeeze(0)
```

Sentence Bert

Charger le modèle

1. Importer la librairie spécifique (complémentaire de transformers)

```
from sentence_transformers import SentenceTransformer
```

2. Charger le modèle pré-entraîné

```
model = SentenceTransformer("paraphrase-multilingual-MiniLM-L12-v2")
```

- Le modèle est récupéré automatiquement depuis le Hub Hugging Face :

 <https://huggingface.co/sentence-transformers/paraphrase-multilingual-MiniLM-L12-v2>

- Réutilisable offline après le premier téléchargement



Hugging Face

Sentence Bert

Obtenir les embeddings

```
# Importer Sentence-BERT
```

```
from sentence_transformers import SentenceTransformer
```

```
# Charger un modèle multilingue optimisé pour similarité
```

```
model = SentenceTransformer("paraphrase-multilingual-MiniLM-L12-v2")
```

```
# Encoder une ou plusieurs phrases
```

```
sentences = [  
    "Le chat dort sur le canapé.",  
    "Un félin se repose sur le sofa."]
```

```
embeddings = model.encode(sentences)
```

```
print(embeddings.shape) # (nombre de phrases=2, dimensions=384)
```

```
print(embeddings) # matrice d'embeddings de phrase
```

`model.encode(liste_de_phrases)` → produit un tableau NumPy (nb_phrases × dim).

`print(embeddings.shape)` → dimensions de la matrice.

`embeddings[i]` → vecteur de la phrase *i*.

BERT/Sentence Bert

Manipuler les embeddings

Similarité cosinus

```
from sklearn.metrics.pairwise import cosine_similarity  
sim = cosine_similarity([embeddings[0]], [embeddings[1]])[0][0]
```



tableaux numpy 2D
ou listes de listes

Distance euclidienne

```
import numpy as np  
dist = np.linalg.norm(embeddings[0] - embeddings[1])
```

Réduction dimensionnelle

```
from sklearn.decomposition import PCA  
pts = PCA(n_components=2).fit_transform(embeddings)
```

Visualisation rapide

```
import matplotlib.pyplot as plt  
plt.scatter(pts[:,0], pts[:,1])
```

A vous de jouer : TP2-SentenceEmb.pynb



CamemBERT (phrases) vs Sentence-BERT : comprendre et comparer les embeddings de phrases

Récupérez le notebook TP2-SentenceEmb.pynb sur Teams

- Partie A — CamemBERT (embeddings de tokens → embeddings de phrase)
 - Afficher les tokens et la matrice des embeddings de tokens initiale (entrée dans le modèle)
 - Visualiser la matrice d'embeddings de position (initiale)
 - Afficher la matrice d'embeddings de tokens (dernière couche)
 - Comparer les matrices initiales (Tokens+Positions) et la matrice finale
 - Evaluer l'évolution des embeddings : comparaison des vecteurs initiaux et finaux (similarité cosinus)
 - Visualiser les embeddings de tokens en 2D (PCA)
 - Générer et visualiser des embeddings de phrase avec 3 méthodes
 - Calculer la similarité Cosinus pour deux paires de phrases

A vous de jouer : TP2-SentenceEmb.pynb



- Partie B — Sentence-BERT (embeddings de phrase optimisés)

- Encoder les mêmes phrases et comparer les similarités cosinus par paires entre SentenceBert et Camembert

- mini-recherche

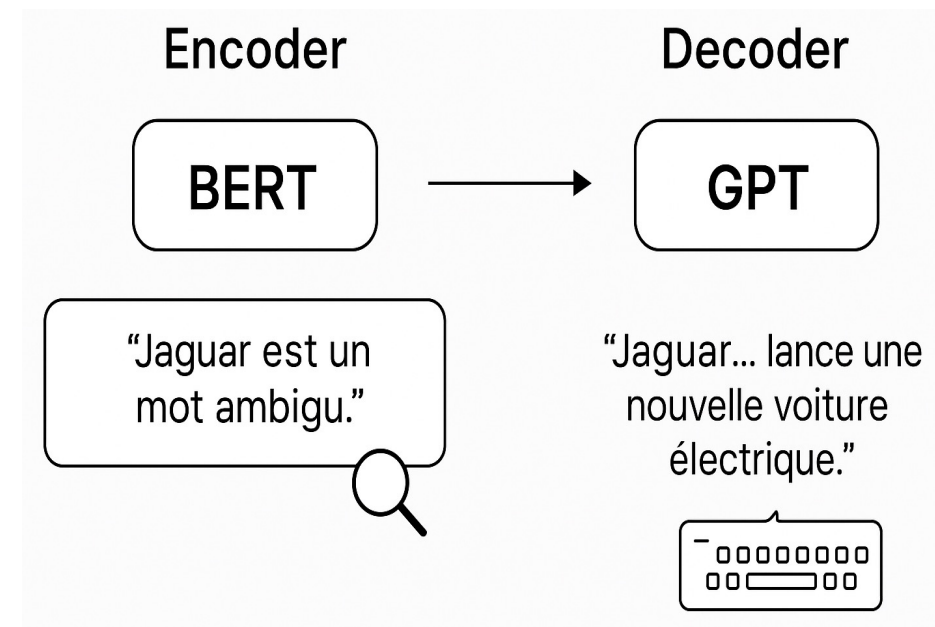
- analyser un code non commenté.
- identifier les indicateurs utilisés pour évaluer la qualité d'un système de recherche sémantique

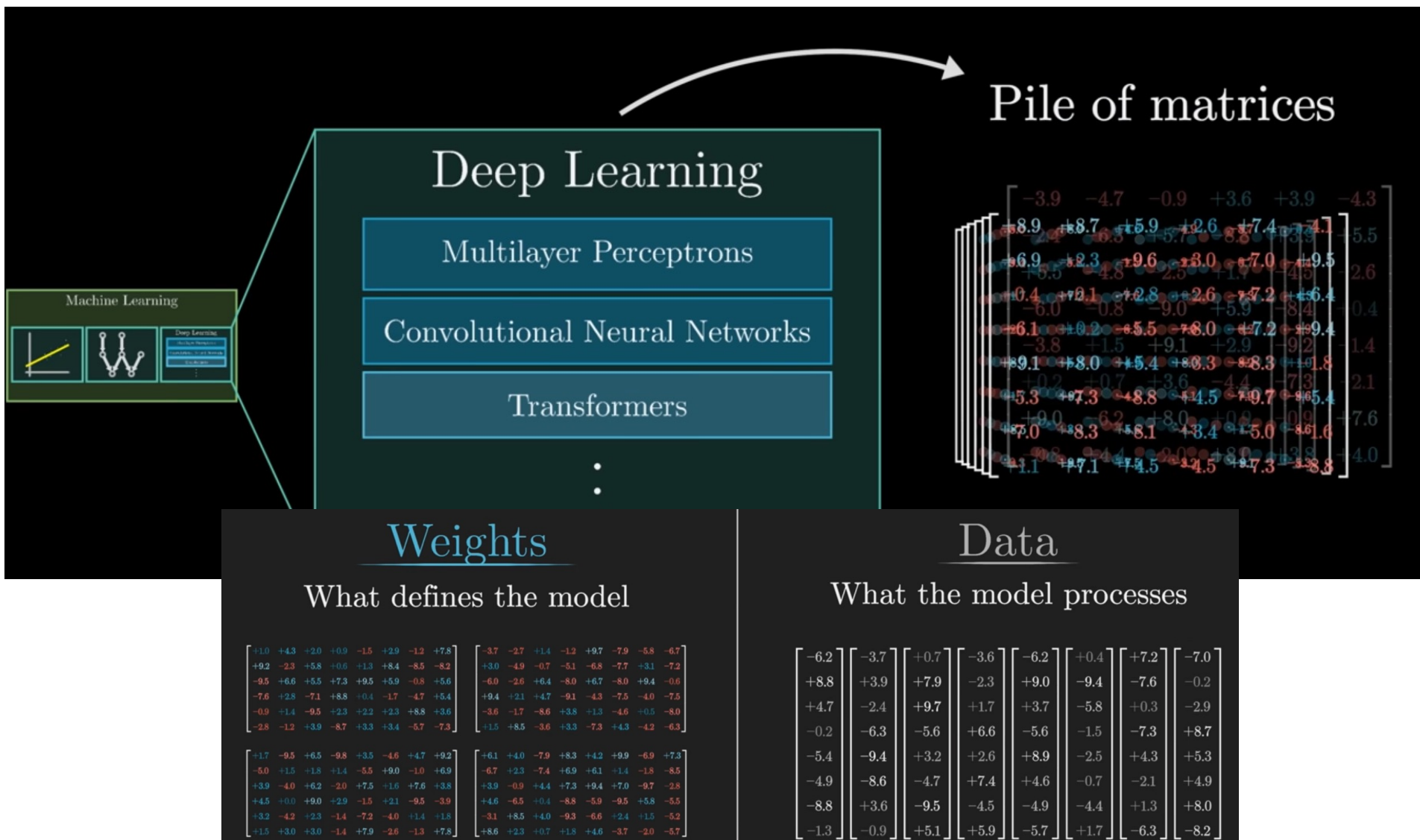


Hugging Face

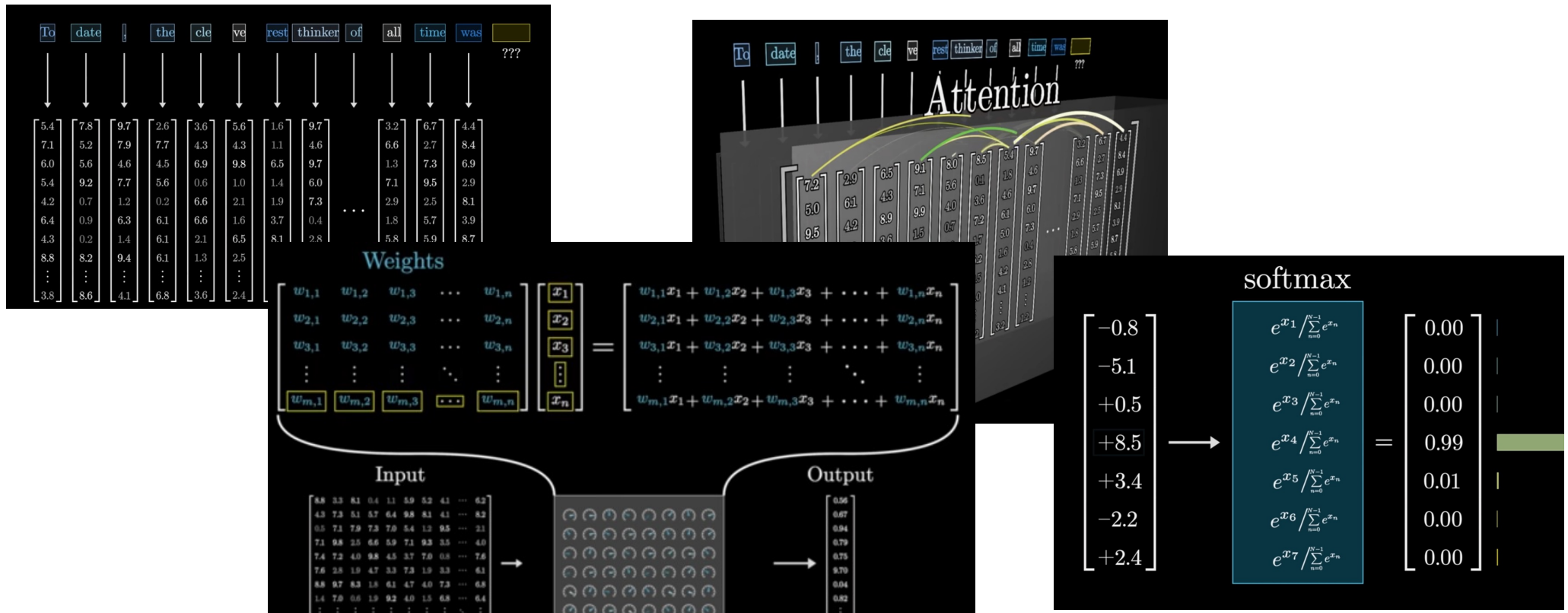
De BERT -> Sentence BERT aux LLM

- Les LLM sont construits sur les mêmes bases (Transformers)
- Ils génèrent aussi des embeddings internes exploitables pour chaque token
- Des vocations différentes :
 - BERT = modèle d'encodage
(vocation= comprendre le langage)
 - Sentence BERT = modèle d'embeddings
(vocation= produire des embeddings de phrase)
 - GPT & LLaMA = modèles génératifs
(vocation= générer du texte)





Etapes d'entrainement des transformers



video sur les transformers: <https://www.youtube.com/watch?v=wjZofJX0v4M>

W_E matrice d'embeddings de GPT3 (Davinci)

Total parameters = $12,288 \times 50,257 = 617,558,016$

50,257 tokens

12,288

\	"	#	\$	%	&	'	(	)	*	...	!	...	Compar	amplification	ominated	regress	Collider	informants	gazed	</endoftext>
-4.0	+1.0	+8.5	+0.4	-4.6	+7.5	-2.5	-9.9	-5.0	-3.6	...	+7.1	-0.8	-1.1	-3.2	+7.5	+8.8	+9.7	-2.4	+9.2	+5.8
+3.5	-5.1	-5.6	-6.6	+8.4	-4.1	-0.9	-0.1	+5.5	+6.8	...	-7.1	-1.4	+6.8	+6.3	-7.9	-6.8	-3.9	-8.4	-1.5	-7.8
+1.4	-5.0	+1.9	-7.6	+9.4	+8.6	-2.1	-5.1	-4.9	-0.3	...	-9.1	+2.8	-1.8	-2.4	+6.1	+4.1	+9.0	-2.9	+7.9	+5.3
-2.8	+2.4	-4.2	+7.4	-7.7	-5.7	-6.3	-1.9	+4.9	+0.5	...	-0.2	-9.9	-1.5	-8.6	-5.8	+8.6	-5.6	+7.1	+6.0	-6.7
+2.1	-7.6	+4.5	+2.7	+6.2	-0.4	+8.2	-8.9	-4.1	+4.3	...	-1.6	-6.5	-7.8	+6.3	-0.5	+7.6	+4.6	-1.8	-2.5	+0.3
+7.7	+4.7	-9.8	+3.8	+8.3	+4.2	-6.4	-0.3	-7.1	-2.8	...	+8.7	+8.4	-4.3	-3.2	+2.0	+9.2	-7.0	-4.8	+7.4	-0.2
+7.9	-6.2	+0.6	-3.4	-3.6	-1.1	-1.3	-2.8	+8.2	+4.6	...	+4.5	-4.2	+1.5	+5.5	+5.9	-3.1	+5.4	+4.7	-7.1	+7.2
-1.2	-0.3	-1.0	+1.3	+2.4	+0.0	+7.3	+2.5	-2.0	-1.6	...	+6.2	-3.0	-5.7	-8.7	+7.4	+8.3	-7.5	-3.3	-6.4	-7.6
⋮	⋮	⋮	⋮	⋮	⋮	⋮	⋮	⋮	⋮	⋮	⋮	⋮	⋮	⋮	⋮	⋮	⋮	⋮	⋮	⋮
+7.9	-8.8	+9.5	-8.0	+7.2	+1.3	-2.6	-3.1	+5.1	-3.7	...	+3.1	+0.3	-0.3	+7.9	+1.1	+6.5	+4.5	-9.1	+5.4	-5.6

Recherche de similarité sémantique : Limites des modèles d'embeddings

Problème 1 : **Scalabilité**

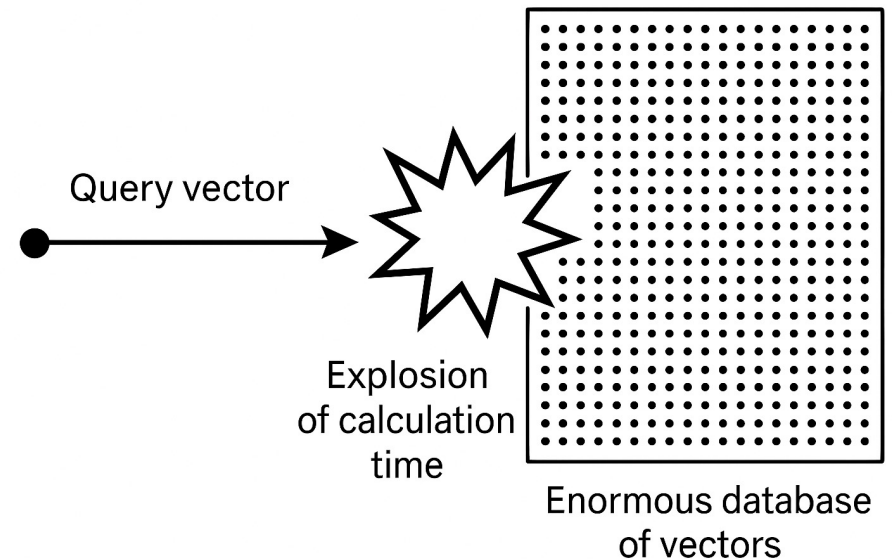
→ comparer un vecteur à des millions d'autres devient trop lent

Problème 2 : **Efficacité mémoire**

→ stockage de vecteurs haute dimension

Problème 3 : **Recherche avancée**

→ besoin de fonctions plus riches qu'un simple “plus proche voisin” (filtrage, métadonnées, mises à jour temps réel)



Nécessité d'outils
supplémentaires

Pour en savoir plus

- Des vidéos particulièrement claires :

- chaine YouTube **3Blue1Brown**

- **cours Neural Networks**

- https://www.youtube.com/playlist?list=PLZHQObOWTQDNU6R1_67000Dx_ZCJB-3pi

- Des explications détaillées sur le fonctionnement d'un transformer (BERT)

- <https://lesdieuxducode.com/blog/2019/4/bert--le-transformer-model-qui-sentraine-et-qui-represente>

